Universitatea Națională de Științe și Tehnologie POLITEHNICA București

APLICAȚII ALE MICROCONTROLERELOR

SISTEM DE PROCESARE DIGITALĂ A SEMNALELOR CU COMANDĂ ARM ȘI EXECUȚIE DSP (AGC, ALE, MF)

ECHIPA NR. 15

Studenți:

Anton George

Mazilu Riana Florentina

Perjaru Adrian

Tudorache Horațiu

Profesor coordonator: Zoican Sorin

1. Cerință și Descrierea Sistemului:

Să se realizeze arhitectura unui sistem format din două subsisteme (ARM și DSP), având rolul de a procesa semnale într-un sistem de comunicații digitale. Sistemul va respecta specificațiile din figura 1.

Microcontrolerul ARM (STM32F103CBT6) gestionează intrările utilizatorului și afișarea stării sistemului, în timp ce microcontrolerul DSP (ADSP2181) efectuează procesarea efectivă a semnalelor.

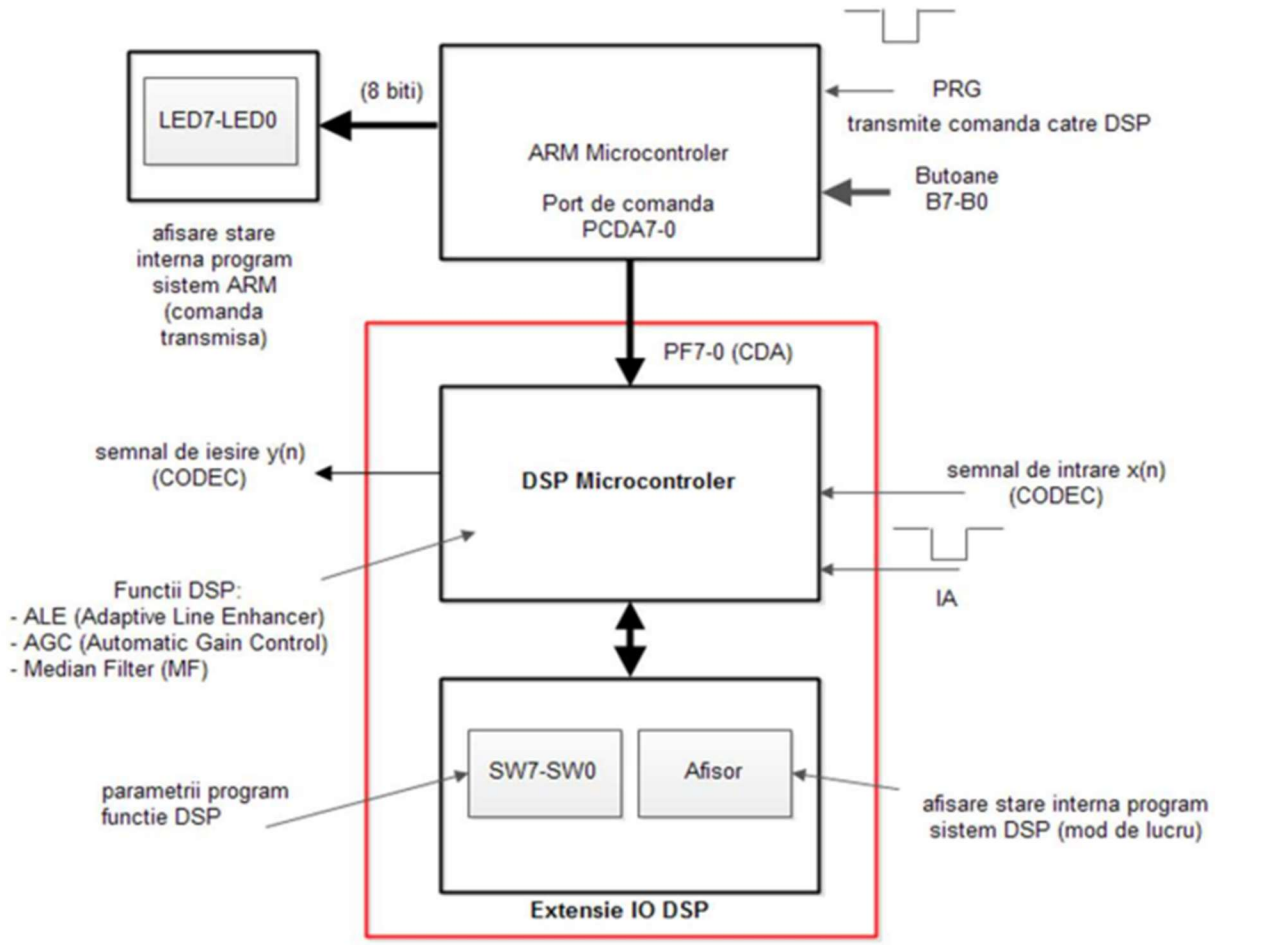


figura 1 – schema bloc a sistemului

Microcontrolerul ARM (STM32F103CBTx): Primește comenzi de la utilizator folosind butoanele B0-B7, care construiesc un cuvânt de comandă pe 8 biți, afișându-l pe LED-urile LED0-LED7 și transmițându-l către DSP prin intermediul portului de comandă PCDA7-0.

Microcontrolerul DSP (ADSP-2181 EZ-Kit Lite): Primește comanda oferită de ARM prin intermediul porturilor PF7-0, o validează și extrage parametrii (prezenți în structura cuvântului de comandă – funcția CDA, ID-ul parametrului, canalul), astfel încât să facă selecția funcției de procesare corespunzătoare. De asemenea, primește semnalul audio de intrare (de forma x(n)) de la CODEC-ul AD1847 și generează semnalul de ieșire (de forma y(n)).

CODEC-ul AD1847 SoundPort: Un convertor ADC/DAC pe 16 biți care are rolul de a digitaliza semnalul audio oferit la intrare și de a reconstrui semnalul la ieșire. Acesta comunică cu DSP-ul prin portul serial SPORT0.

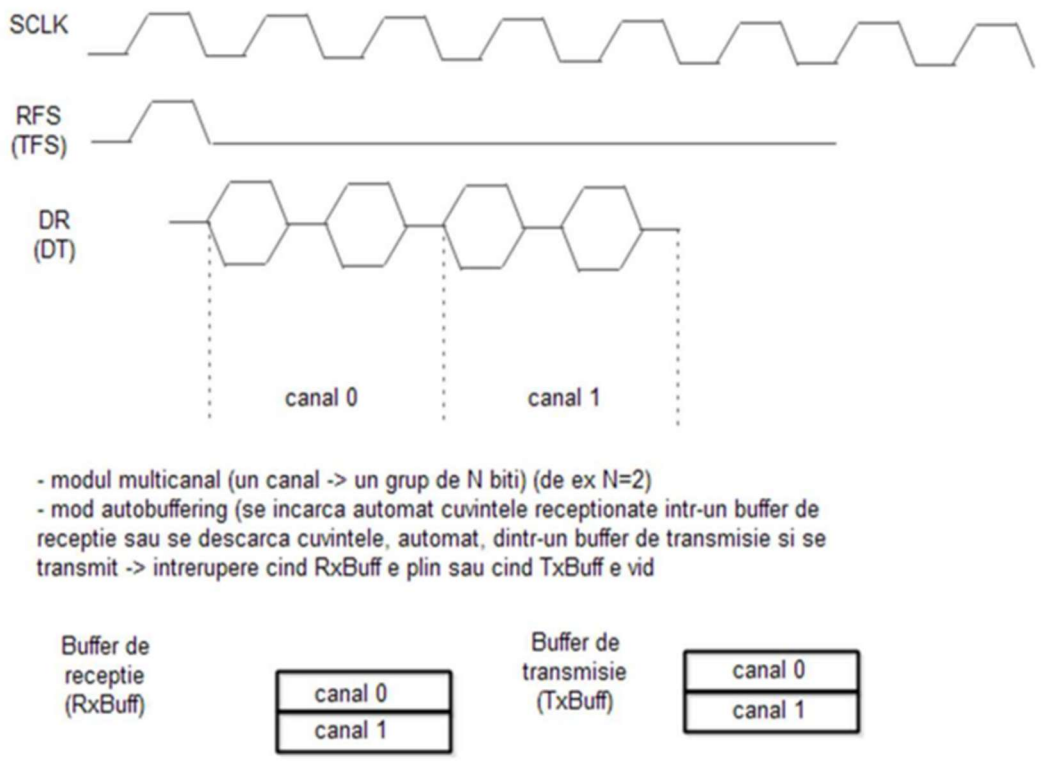


figura 2 – conectarea DSP prin portul serial SPORT0

EXTENSIA IO DSP: Aceasta include comutatoarele SW7-0 pentru configurarea manuală a parametrilor și un afișor folosit pentru a monitoriza stările interne ale DSP-ului.

2. Descrierea funcționalității porturilor de intrare și ieșire:

A) ARM

Microcontrolerul ARM acționează ca element de decizie și interfață cu utilizatorul, având rolul de a configura și coordona activitatea procesorului de semnal (DSP).

Intrările sistemului ARM:

Configurarea funcționării se realizează prin două mecanisme digitale:

- **butonul PRG (Trigger Comandă)** – este un *buton fără reținere* care, la apăsare, generează evenimentul de transmitere a pachetului de control către DSP
- **grupul de butoane B7-0 (Parametrii de Stare)** – acesta definește *cuvântul de control* care va fi trimis către DSP

Structura Cuvântului de Control (Magistrala PCDA7-0):

Conform schemei de soluție propuse, configurația biților este următoarea:

BIT	SEMNIFICAȚIE	DETALII
D7	1	Bit de validare/sincronizare.
D6	1	Bit de validare/sincronizare.
D5-D4	CDA1-CDA0	Selector pentru algoritmul DSP: ALE, AGC sau MF.
D3	CANAL	Selector pentru canalul de date.
D2-D0	ID2-ID0	Identificatorul DSP-ului țintă (permite adresarea individuală într-un sistem cu mai multe DSP-uri).

Ieșirile sistemului ARM (Magistrala PCDA7-0):

ARM-ul comunică starea sistemului prin două căi principale:

- **interfața locală (LED7-0)** – aceste LED-uri sunt conectate direct la ARM și reflectă *starea internă* a programului. Ele confirmă vizual utilizatorului că pachetul de comandă a fost procesat și transmis corect
- **magistrala de comandă (PCDA7-0)** – reprezintă *portul de ieșire pe 8 biți* prin care ARM-ul transmite instrucțiunile către intrarea DSP-ului, PF7-0 (CDA)

B) DSP

Gestionarea Parametrilor din Switch-uri (SW7-SW0):

Logica Semnalului de Întrerupere (IA – Interrupt Acknowledge):

DSP-ul nu rulează codul la întâmplare, el este sincronizat cu CODEC-ul audio prin semnalul IA.

1. **Eșantionare (x(n)):** *CODEC-ul transformă sunetul analogic în digital* și ridică semnalul IA (un impuls de tipul ”low-pulse”).
2. **Întrerupere:** *DSP-ul detectează IA*, oprește orice altă activitate și citește valoarea x(n) din registrul de intrare.
3. **Procesare:** Se aplică algoritmul ales (ALE, AGC sau MF).
4. **Ieșire (y(n)):** Valoarea rezultată este scrisă în buffer-ul de ieșire.

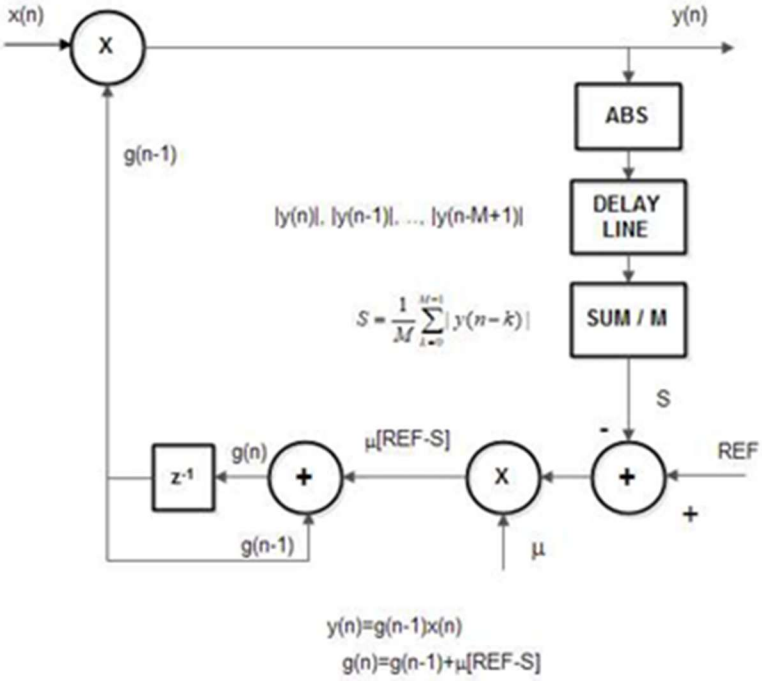
3. Descrierea funcțiilor de prelucarea digitală a semnalului:

Descrierea teoretică a funcțiilor folosite este prezentată mai jos. În etapa actuală, *toate funcțiile realizează bypass (y(n) = x(n))*.

3.1. Automatic Gain Control (AGC).

Funcția are rolul de a menține nivelul semnalului de la ieșire relativ constant, indiferent de variațiile nivelului semnalului de la intrare. Sistemul calculează valoarea medie a modulului semnalului de la ieșire (S) pe o fereastră de M eșantioane. Această medie este comparată cu o valoare de referință (REF).

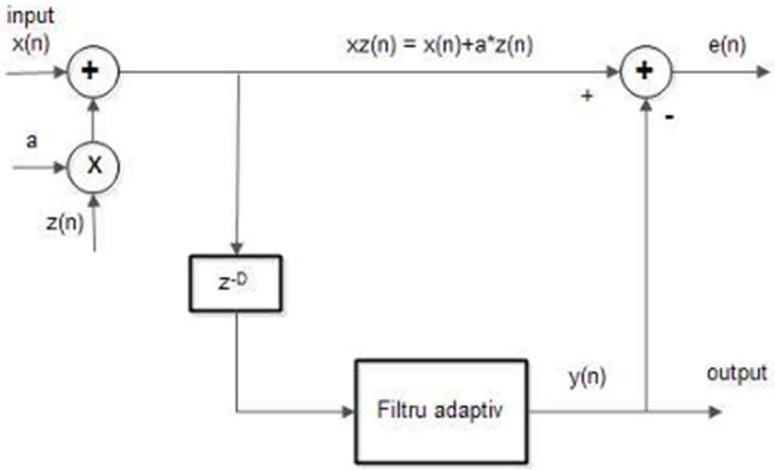
Parametri: REF, M, μ



3.2. Adaptive Line Enhancer (ALE).

Funcția este utilizată pentru a extrage sau a amplifica semnale periodice care sunt expuse unui zgomot de bandă largă. Se folosește un filtru adaptiv și o linie de întârziere (de întârziere D) aleasă astfel încât zgomotul (aleatoriu) să devină necorelat, în timp ce partea periodică a semnalului să rămână corelată.

Parametri: k, a, μ, λ

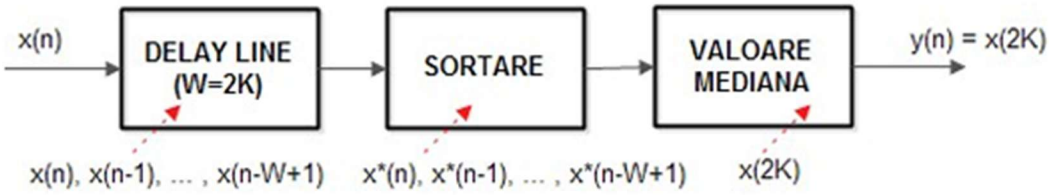


$$y(n) = \sum_{k=0}^{N-1} h(k).xz(n-k-D)$$

$$h(k) = (1-\mu.\lambda).h(k) + \mu.xz(n-k-D).e(n) \quad k = 0,1,\ldots,N-1$$

3.3. Median Filter (MF).

Acesta este un filtru neliniar, extrem de eficient în eliminarea zgomotului de timp impuls (vârfuri bruște de scurtă durată). Se preia o fereastră de W = 2K eșantioane din semnal (unde K – dimensiunea ferestrei de filtrare), eșantioane ce vor fi sortate crescător. Valoarea de la ieșire (semnalul y(n)) va fi astfel valoarea mediană (cea din mijlocul listei sortate).



Parametru: K

4. Schema electrică a subsistemului ARM:

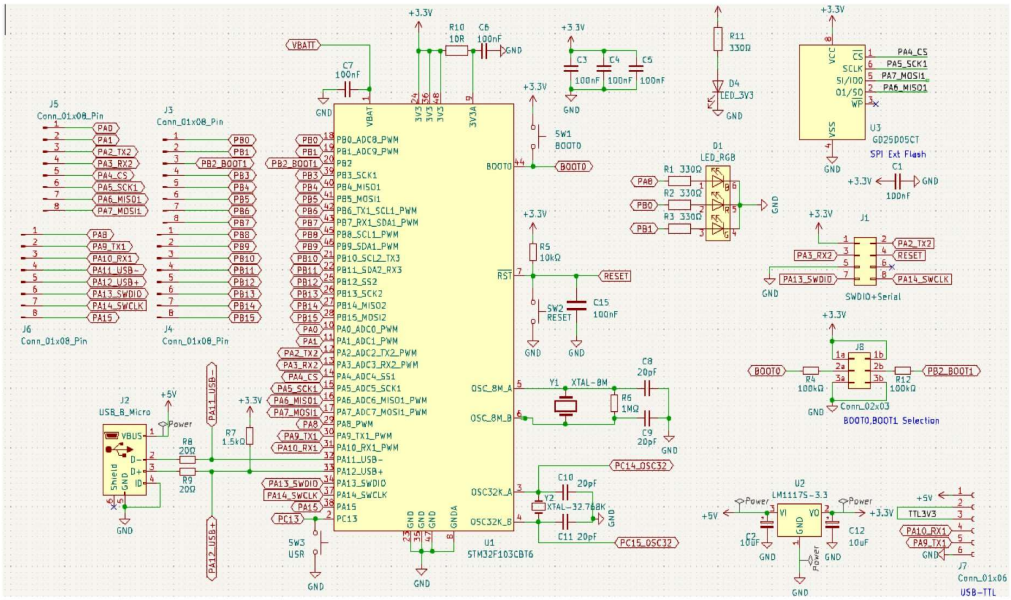


figura 3 – schema electrică STM32

Placa STM32 folosește structura microcontrolerului STM32F103CBT6. Caracteristicile acesteia sunt:

- **J3,J4,J5,J6 – GPIO Headers** = conectori de expansiune pentru pinii GPIO ai microcontrolerului, aceștia pot fi folositi pentru senzori, comunicatie
- **J2 – USB Micro-B** = conectare la PC pentru programare/alimentare -> D+, D- trec prin R7, R8 pentru compatibilitatea USB (protecție ESD)
- **J1** = conector de 8 pini care combina interfața de debug/programare SWD (serial wire debug) și interfata UART (serială)
- **PA2_TX2** = UART Tx (iesire din STM32 către PC)
- **PA3_RX2** = UART Rx (intrare de la PC catre STM32)
    - **PA13_SWDIO** = linie SWDIO date/debug (esential pentru STM32CUBEIDE - upload din PC trimite firmware prin usb la st-link, iar st-link trimite date pe SWDIO si sincronizeaza prin SWCLK, iar STM32 primeste codu si il scrie in Flash)
    - **PA14_SWCLK** = linie SWCLK ceas debug

U2 = regulator de tensiune

- *intrare de +5V - iesire +3.3V* pentru alimentarea microcontrolerului
- *C12+C13* stabilizează tensiunea de iesire
- **D2 - Dioda Schottky** = protejează circuitul la conectarea inversa a alimentării
- **Y1** = oscilator principal de 8MHz
- **Y2** = oscilator RTC de 32.768kHz (condesatoarele si rezistentele asigura functionare stabila)

U3 = Memorie SPI Flash externa

- **memorie non-volatila** (nu isi pierde continutul atunci cand se intrerupe alimentarea cu energie) conectata pe SPI
- stocare de date suplimentare
- include pin WP (write protect) si HOLD

STM32 are o memorie Flash interna (non-volatila) pentru salvarea codului - firmware.

- **memoria suplimentara** = SPI Flash (Serial Peripheral Interface) = protocol de comunicatie seriala sincronă
- **MOSI** = Master Out Slave in
- **MISO** = Master In Slave Out

BOOT0 = determina sursa de boot(Flash/RAM/UART (J8 determina selectia))

- **SW2** - RESET manual (R5 rezistență de pull-up)

5. Schema electrică a subsistemului DSP:

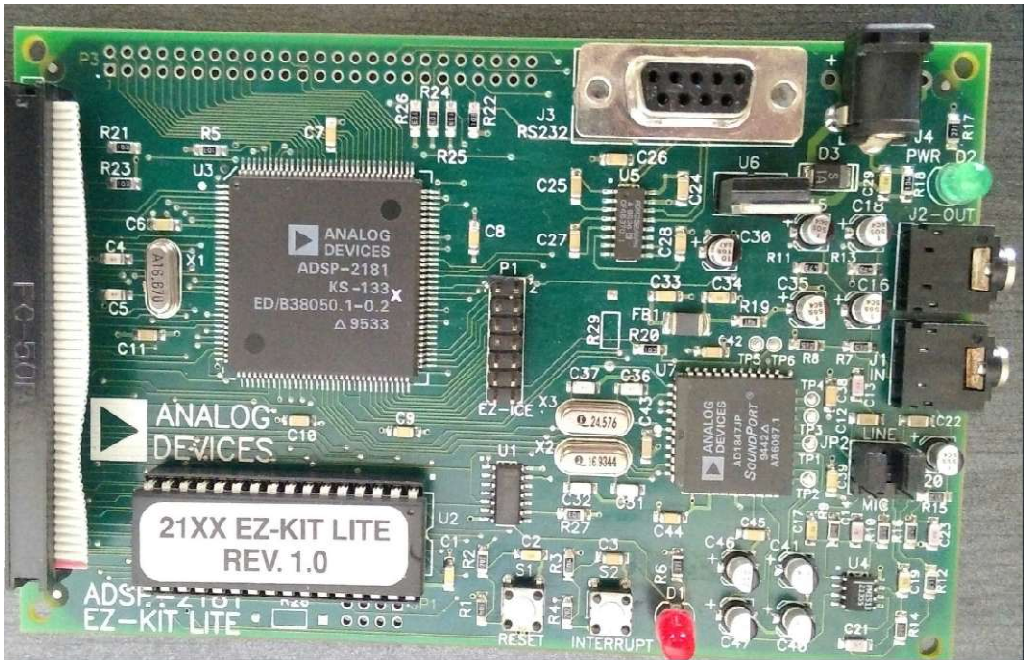


figura 4 – placa de evaluare ADSP-2181 EZ-Kit Lite (*vedere fizică*)

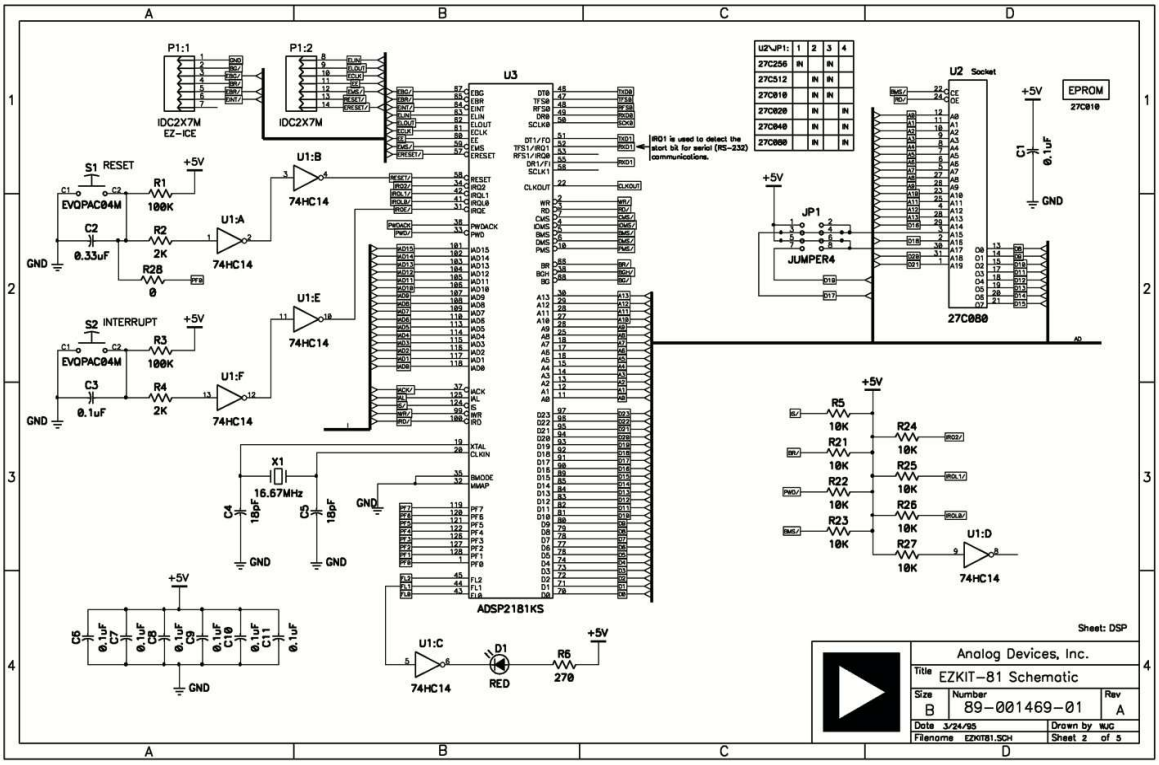


figura 5 – schema electrică EZKIT-81 – DSP

CPU (ADSP2181KS)

- gestionează toate operațiunile digitale
- nucleu optimizat pentru calcule rapide (RISC)
- magistrale de date și adrese

P1, P2 - interfața de depanare EZ-ICE

- port pentru emulatorul EZ-ICE (depanarea codului direct pe Hardware)

S1 RESET - circuit de reset

- reset hardware pentru DSP prin apasarea lui S1
- 74HC14 - inversor Schmitt Trigger (asigura stabilitatea semnalului de reset)
- R1 menține linia de reset la +5V (HIGH)
- C2 creeaza o scurta intarziere (S1 apăsat)
- C2 se incarcă prin R1 (S1 *eliberate*)

S2 INTERRUPT - circuit de întrerupere externa

- activarea interruperii externe prin apasarea S2
- nu consuma constant resurse
- C3 => filtru in trecerile HIGH-LOW

X1 - oscilator

- frecventa de 16.67 Mhz
- semnal de tact pentru DSP
- C4 si C5 stabilizeaza oscilatorul
- semnalul dat de X1 este transmis la CLKIN

C6, C7, C8, C9, C10, C11 - banc de condensatoare de decuplare

- filtrarea zgomotului si stabilizarea tensiunii de alimentare
- elimina zgomotele de inalta frecventa
- reduce interferentele electromagnetice

A0 – A14 - linii de adresa pentru selectarea memoriei

WR, RD - linii de control pentru citire/scriere in memorie

R5, R21, R24, R22, R25, R23, R26, R27 - circuit de Pull-Up pentru pinii de control

- rezistentele mențin starea logica HIGH pentru anumite semnale (SR, PWDR, IRQ)
- U1:D foloseste aceste semnale pentru a asigura stabilitatea și a preveni starile necontrolate

U2 Socket - memorie EPROM

- nevolatila, folosită pentru a stoca programul firmware al DSP-ului
- selectata prin JP1 - permite configurarea dimensiunii memoriei
- CE - Chip Enable (activeaza memoria)
- OE - Output Enable (activeaza ieșirea de date)
- A0-A19 - magistrala de adrese (2^20 locatii de memorie)
- O0-O7 - output data

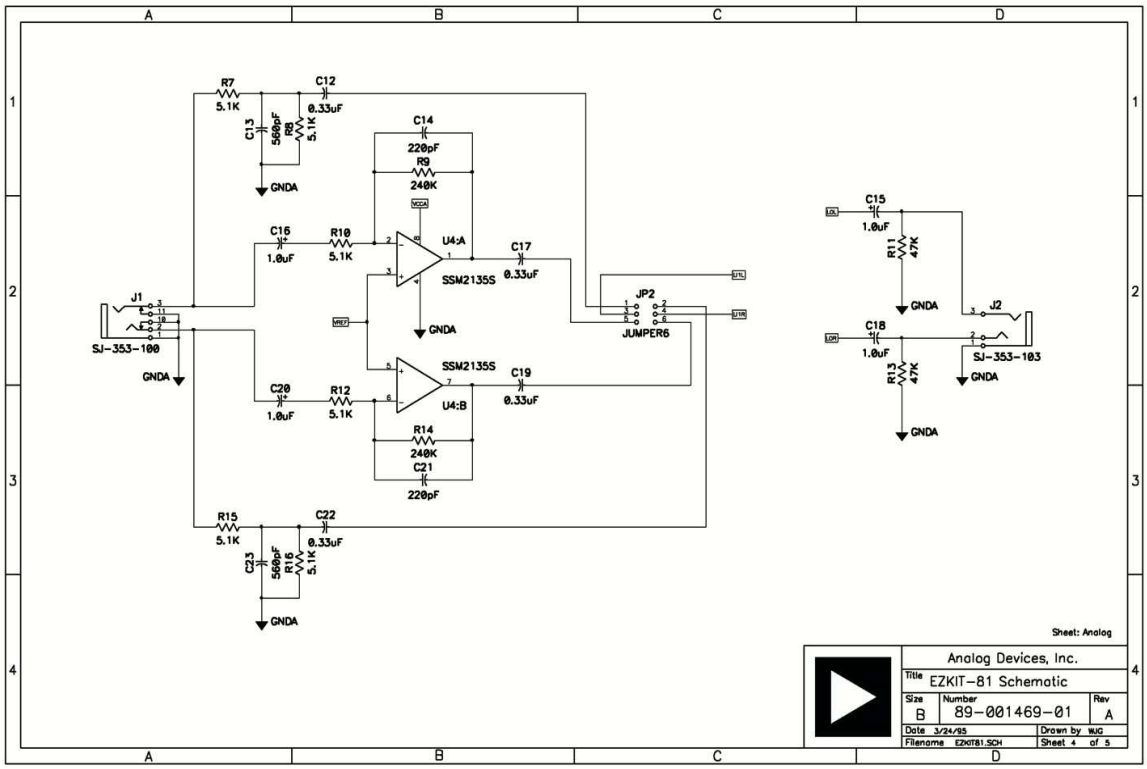


figura 6 – schema electrica EZKIT-81 – CODEC

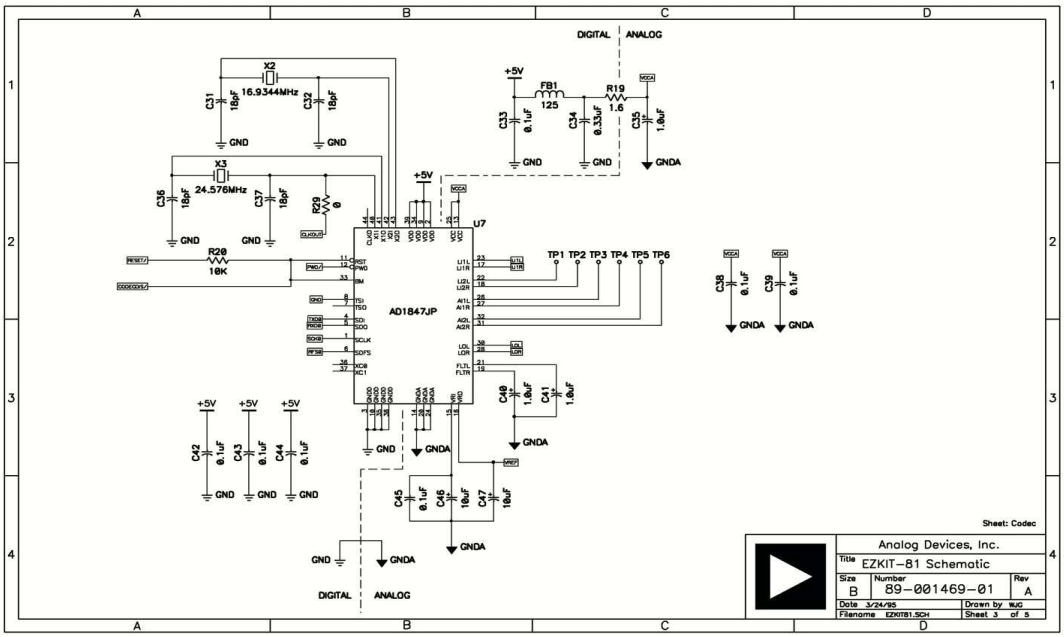


figura 7 – schema electrica EZKIT-81 – CODEC

AD1847JP = CODEC audio-stereo

- face conversia dintre un semnal digital (folosit de DSP) si un semnal analogic (căști/difuzoare)

L1R, L1L - intrări audio stânga/dreapta

L2R, L2L - a doua pereche de intrări audio

A1L, A2R - linii pentru microfoane sau alte surse audio

LOL, LOR - ieșiri audio analogice (difuzoare, casti)

- FLTR - filtru analogic

X2, X3 - Memorie EPROM

- nevolatilă, folosita pentru a stoca programul firmware al DSP-ului
- selectată prin JP1 - permite configurarea dimensiunii memoriei
- CE - Chip Enable (activează memoria)
- OE - Output Enable (activează iesirea de date)
- A0-A19 - magistrala de adrese (2^20 locatii de memorie)
- O0-O7 - output data

RESET

- mentinut la HIGH prin R20

TXD0, RXD0, SCK0, RFS0 - linii de control pentru interfata digitala

- SDI = Serial Data Input - intrare de date digitale (de la DSP)
- SDO = Serial Data Output - ieșire de date digitale (catre DSP)
- SCLK = Serial Clock - semnal de sincronizare pentru transmitere date
- SOFS = Sync Frame – indică inceputul unui pachet de date audio

Filtru de separare intre alimentarea digitală si cea analogica

- FB1 = ferrite bead - blocheaza zgomotul de inalta frecventa inainte sa ajunga la partea analogica
- C35 actioneaza ca un rezervor de energie (tensiune stabila)

Filtru de stabilizare pt tensiunea de referinta VREF

- stabilizează tensiunea de referinta necesara in conversia analog-digital si digital-analog
- C45 filtreaza frecvențe inalte
- C46 si C47 - filtrare frecvenței medii și reglare a VREF

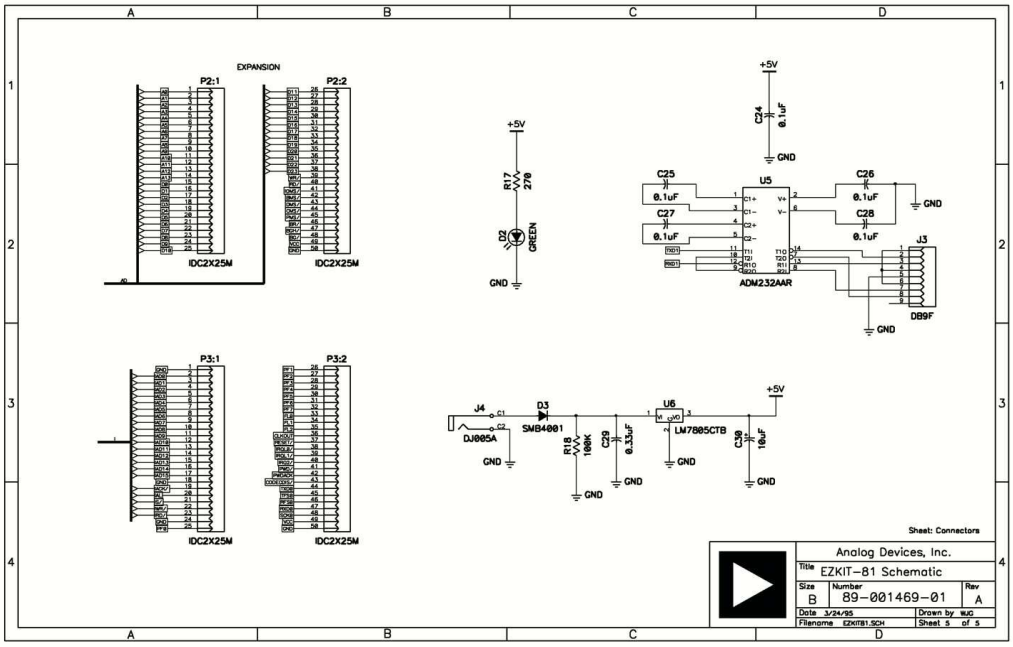
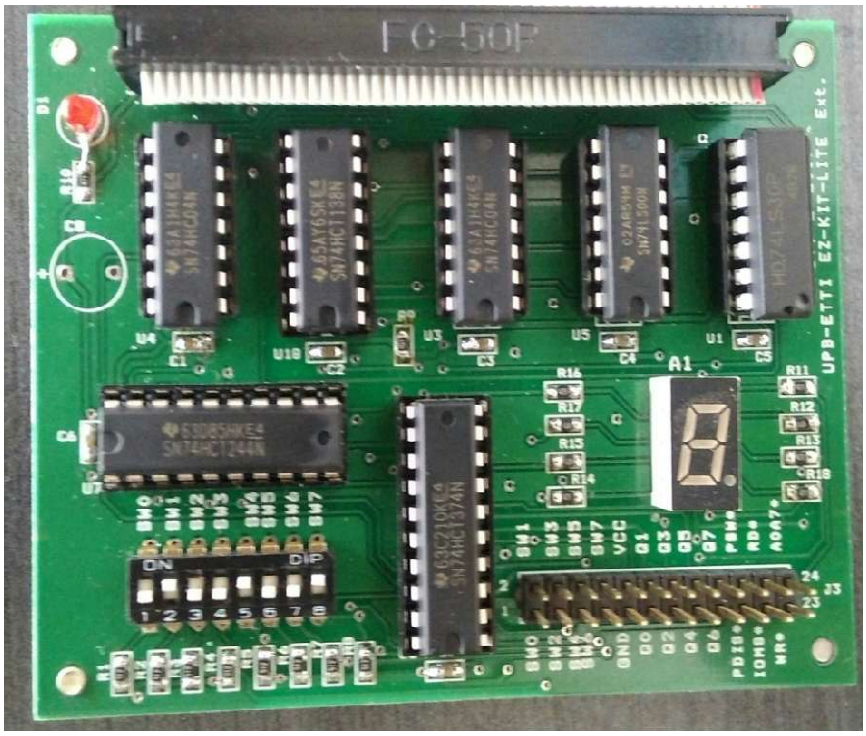


figura 8 – schema electrica EZKIT-81 – Conectori

6. Schema electrică a extensiei IO:



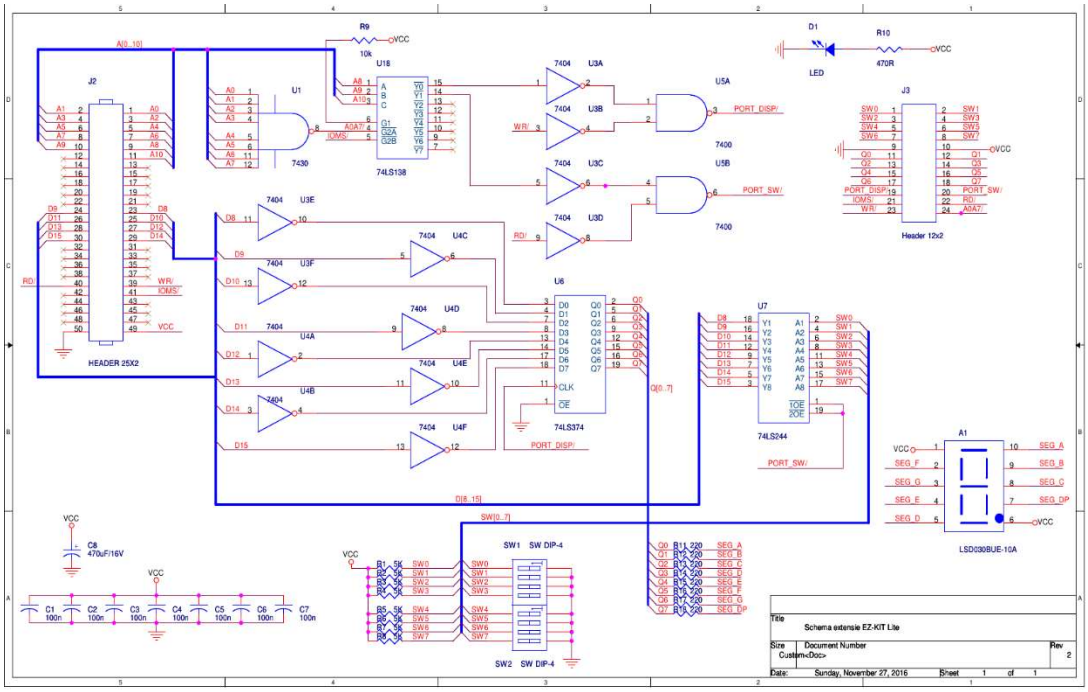


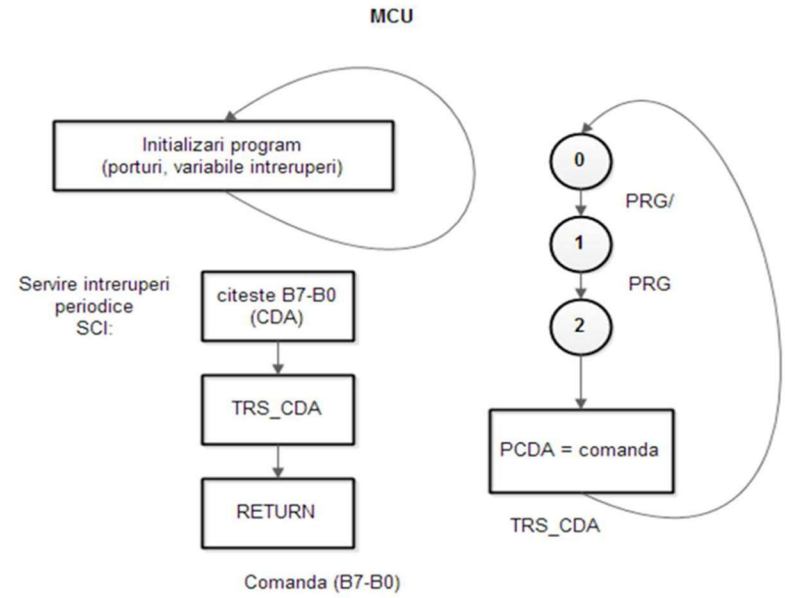
figura 9/10 – placa de extensie IO (*vedere fizică*)/schema electrica a plăcii de extensie IO

Circuite Integrate (IC-uri) și Componente:

- **7404 (U3, U4):** Porți **NOT** (inversoare). Invertesc semnalele de control și adresele pentru a asigura polaritatea corectă necesară activării celorlalte chip-uri.
- **7430 (U1):** Poartă **NAND cu 8 intrări**. Verifică liniile de adresă superioare; generează o ieșire 'low' doar când adresa selectată corespunde acestui modul de extensie.
- **74LS138 (U18): Decodor 3-la-8 linii**. Selectează sub-modulele extensiei (afișajul sau butoanele) prin activarea unei singure ieșiri în funcție de adresele A8, A9 și A10.
- **74LS374 (U6): Registru octal cu latch**. Stochează datele de pe magistrală pentru a menține aprinse segmentele afișajului LED, chiar și după ce procesorul trece la altă sarcină.
- **74LS244 (U7): Buffer octal (Tri-state)**. Acționează ca o poartă care permite trecerea stării butoanelor către procesor doar atunci când acesta execută o comandă de citire (PORT_SW/).
- **7400 (U5):** Porți **NAND cu 2 intrări**. Combină semnalele de selecție cu semnalele de scriere/citire (WR/, RD/) pentru a genera semnalele finale de activare a porturilor.
- **Afișaj LED (A1 - LSD030BUE):** Afișor cu 7 segmente plus punct decimal. Primește datele stocate de U6 prin rezistențe de limitare pentru a afișa cifre sau caractere.
- **Comutatoare (SW1, SW2):** Două blocuri de câte 4 switch-uri (DIP-4) care permit setarea manuală a 8 biți de date (SW0-SW7) pentru a interacționa cu programul DSP-ului.

7. Subsistemul ARM – descriere conceptuală/funcțională

7.1. Descrierea mașinii de stări TRS_CDA



D7	D6	D5	D4	D3	D2	D1	D0
1	1	CDA1	CDA0	CANAL	ID2	ID1	ID0

PRG - buton fara retinere

STARE 0 – ELIBERARE PRG: Se verifică dacă butonul PRG (PA9) este eliberat. Dacă da, se trece în starea 1. NU se va accepta o comandă nouă cât butonul este ținut apăsat.

STARE 1 – APĂSARE PRG: La eliberarea butonului PRG, se vor citi butoanele B0-B7 (shift dreapta cu 1, mască 0x3F pentru 6 biți). Se construiește comanda: *comanda_dsp = 0xC0 | date_butoane*. Se trece in starea 2.

STARE 2 – TRANSMITERE COMANDĂ: Se va transmite comanda pe magistrala PCDA către DSP (PB8-PB15). Se va afișa aceeași comandă pe LED-uri (PB0-PB7). Se revine în starea 0.

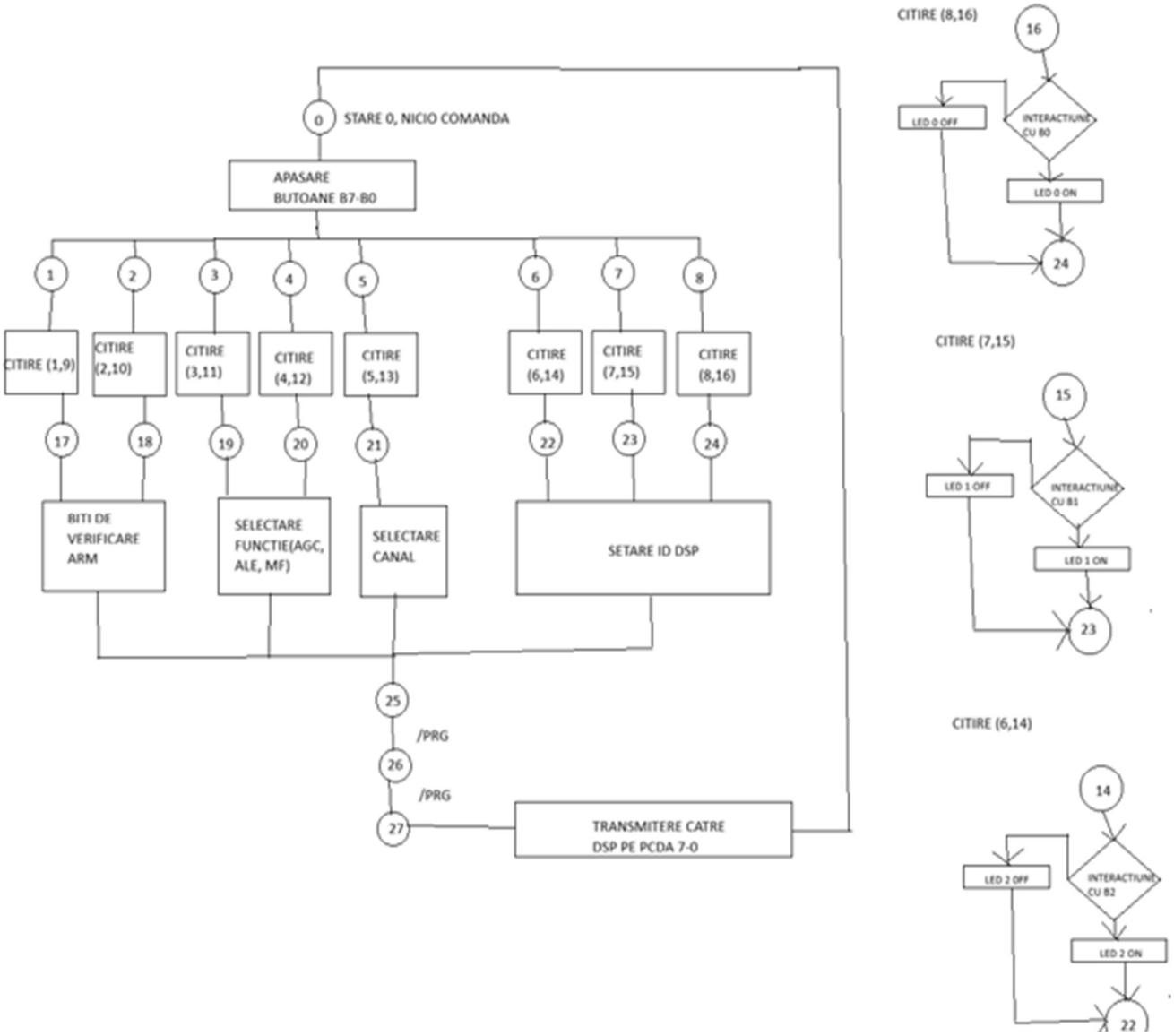


figura – stări ARM elaborate

7.2. Configurare Hardware (STM32CubeMX)

- **MCU:** STM32F103CBTx (LQFP48, Cortex-M3)
- **Clock:** HSE 8 MHz → PLL ×9 → SYSCLK = 72 MHz, HCLK = 72 MHz, APB1 = 36 MHz, APB2 = 72 MHz
- **Timer TIM2:** Clock intern 72 MHz, PSC = 7199, ARR = 100, Counter Up → frecvența de întrerupere ≈ 99 Hz (~10 ms)
- **Debug:** Serial Wire (SWD)
- **GPIO intrări (fără pull-up/pull-down):** PA1–PA6 = butoane B0–B5, PA7 = B6, PA8 = B7, PA9 = PRG
- **GPIO ieșiri (push-pull, low speed):** PB0–PB7 = LED0–LED7, PB8–PB15 = PCDA0–PCDA7
- **NVIC:** TIM2 global interrupt activat, prioritate 0

CODUL C PENTRU STM32:

```
/* USER CODE BEGIN Header */
/**
*****************************************************************************
 * @file        : main.c
 * @brief       : Main program body
*****************************************************************************
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
```

```
**********************************************************************
*****
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */
/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim2;

/* USER CODE BEGIN PV */
uint8_t comanda_dsp = 0;      // comanda de 8 biti transmisa catre DSP
uint8_t stare_trs = 0;        // starea masinii de stari TRS_CDA (0, 1, 2)
/* USER CODE END PV */
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
/* Timer2 Interrupt Service Routine - Masina de stari TRS_CDA */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
```

```
{
  if (htim->Instance == TIM2)
  {
    // Citeste starea curenta a butonului PRG (PA9)
    GPIO_PinState prg = HAL_GPIO_ReadPin(GPIOA, PRG_Pin);

    switch (stare_trs)
    {
      case 0:
        // STAREA 0: Asteptam ca PRG sa fie ELIBERAT
        // Scopul: ne asiguram ca butonul a fost eliberat
        // inainte de a accepta o noua comanda
        if (prg == GPIO_PIN_RESET)
        {
          stare_trs = 1;  // PRG eliberat -> trecem in starea 1
        }
        // Daca PRG e inca apasat, ramanem in starea 0
        break;

      case 1:
        // STAREA 1: Asteptam ca PRG sa fie APASAT
        // Cand PRG e apasat, citim butoanele si construim comanda
        if (prg == GPIO_PIN_SET)
        {
          // Citeste B0-B5 din PA1-PA6 (6 biti)
          uint16_t stare_port_A = GPIOA->IDR;
          uint8_t date_butoane = (uint8_t)((stare_port_A >> 1) & 0x3F);

          // Construieste comanda: D7=1, D6=1, D5-D0=butoane
          comanda_dsp = 0xC0 | date_butoane;

          stare_trs = 2;  // Comanda pregatita -> trecem in starea 2
        }
        // Daca PRG nu e apasat, ramanem in starea 1
        break;
```

```
    case 2:
      // STAREA 2: Transmitere comanda pe PCDA si afisare pe LED-uri
      // Octetul superior (PB8-PB15) = PCDA7-PCDA0 -> catre DSP
      // Octetul inferior (PB0-PB7) = LED7-LED0 -> afisare comanda
      GPIOB->ODR = ((uint16_t)comanda_dsp << 8) | (uint16_t)comanda_dsp;

      stare_trs = 0;  // Revenim in starea 0 (asteptam eliberare PRG)
      break;
    default:
      stare_trs = 0;  // Stare invalida -> reset la 0
      break;
   }
  }
}
/* USER CODE END 0 */
/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */
  /* USER CODE END 1 */
  /* MCU Configuration--------------------------------------------------------*/
  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();
  /* USER CODE BEGIN Init */
  /* USER CODE END Init */
  /* Configure the system clock */
  SystemClock_Config();
  /* USER CODE BEGIN SysInit */
  /* USER CODE END SysInit */
  /* Initialize all configured peripherals */
  MX_GPIO_Init();
```

```
  MX_TIM2_Init();
  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2); // Pornește Timer 2 în regim de întreruperi
  /* USER CODE END 2 */
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE; // Folosim
cristalul extern
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9; // 8MHz * 9 = 72MHz
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) { Error_Handler(); }

  RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
```

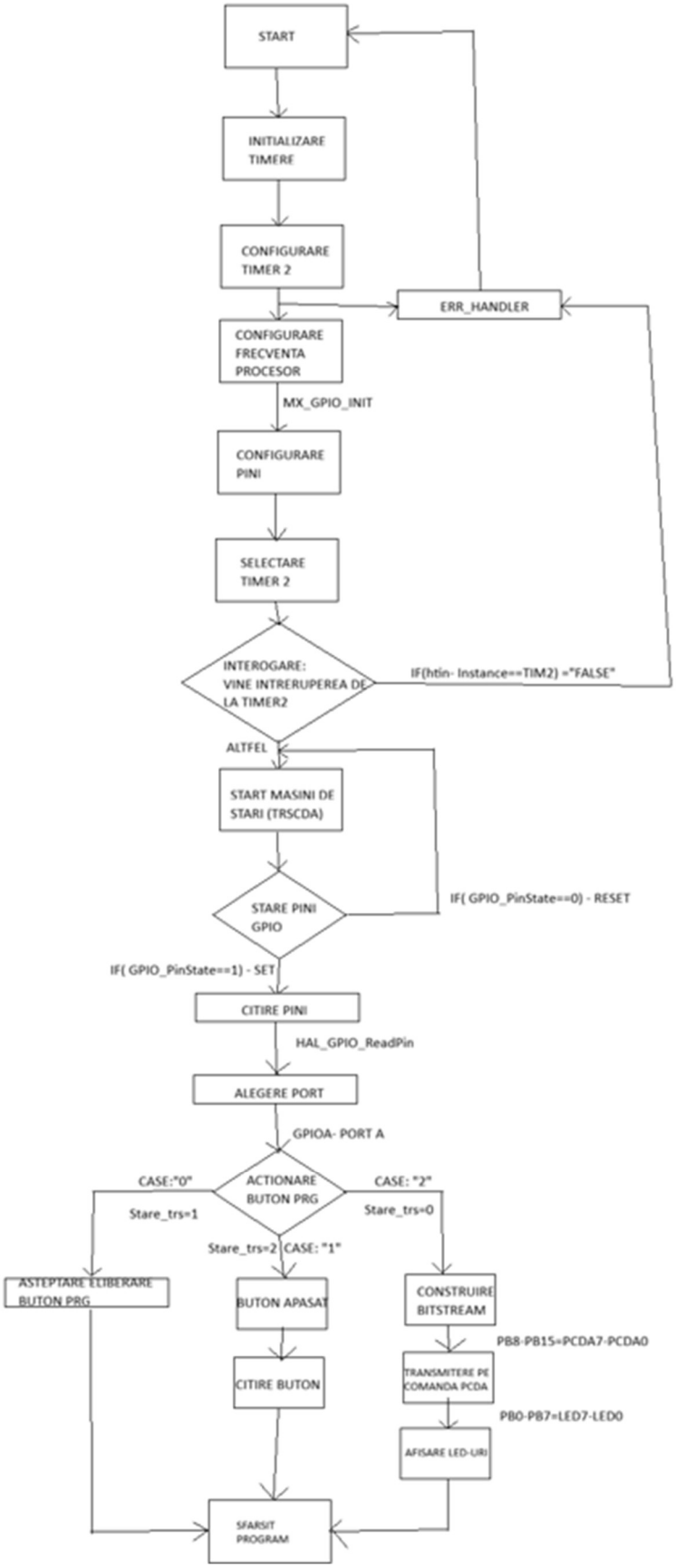
```
                    |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
HAL_OK) { Error_Handler(); }
}
/**
  * @brief TIM2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM2_Init(void)
{

  /* USER CODE BEGIN TIM2_Init 0 */
  /* USER CODE END TIM2_Init 0 */
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  /* USER CODE BEGIN TIM2_Init 1 */
  /* USER CODE END TIM2_Init 1 */
  htim2.Instance = TIM2;
  htim2.Init.Prescaler = 7199;
  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim2.Init.Period = 100;
  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
```

```
  {
    Error_Handler();
  }
  if (HAL_TIM_OC_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_TIMING;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_OC_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM2_Init 2 */
  /* USER CODE END TIM2_Init 2 */
  HAL_TIM_MspPostInit(&htim2);
}
/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
```

```
/* USER CODE BEGIN MX_GPIO_Init_1 */
/* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin
                    |PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin
                    |PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin
                    |LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin, GPIO_PIN_RESET);
  /*Configure GPIO pins : B0_Pin B1_Pin B2_Pin B3_Pin
                    B4_Pin B5_Pin B6_Pin B7_Pin
                    PRG_Pin */
  GPIO_InitStruct.Pin = B0_Pin|B1_Pin|B2_Pin|B3_Pin
                    |B4_Pin|B5_Pin|B6_Pin|B7_Pin
                    |PRG_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
  /*Configure GPIO pins : LED0_Pin LED1_Pin LED2_Pin PCDA2_Pin
                    PCDA3_Pin PCDA4_Pin PCDA5_Pin PCDA6_Pin
                    PCDA7_Pin LED3_Pin LED4_Pin LED5_Pin
                    LED6_Pin LED7_Pin PCDA0_Pin PCDA1_Pin */
  GPIO_InitStruct.Pin = LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin
                    |PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin
                    |PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin
                    |LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
```

```
}
/* USER CODE BEGIN 4 */
/* USER CODE END 4 */
/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state
*/
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *        where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line
number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

figura – schemă explicativă a codului STM



7.3. Simulări în programul STM32CubeIDE

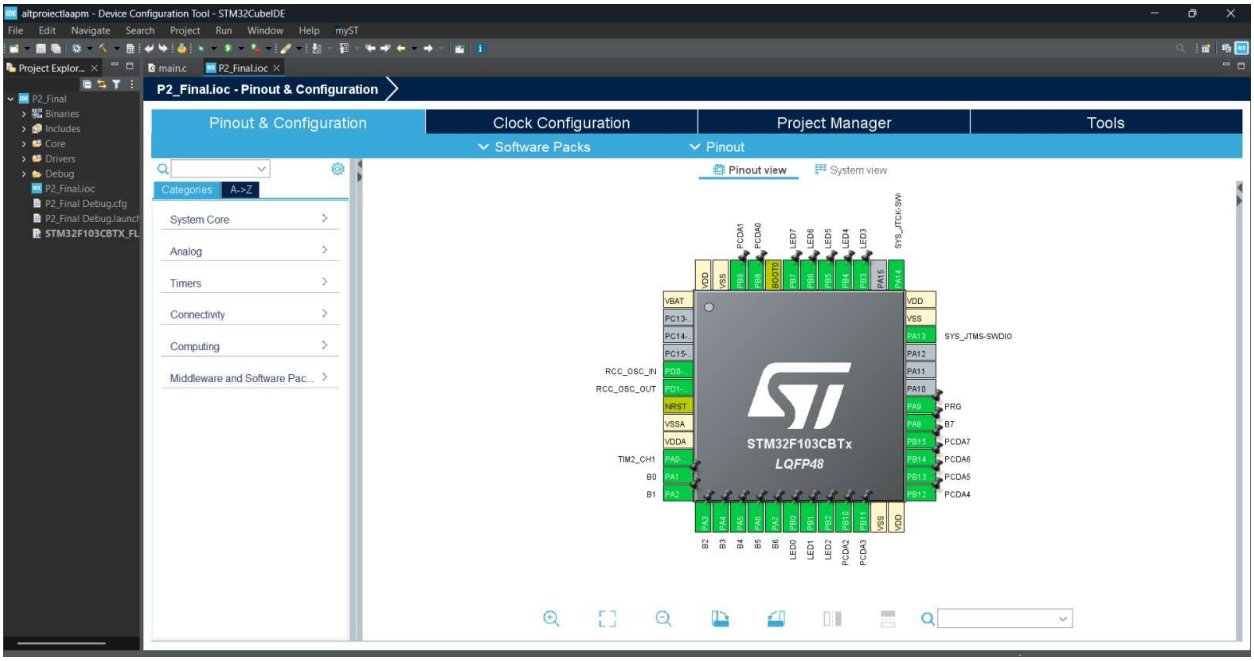


figura 11 – configurarea pinilor (pinout view) – STM32CUBEMX

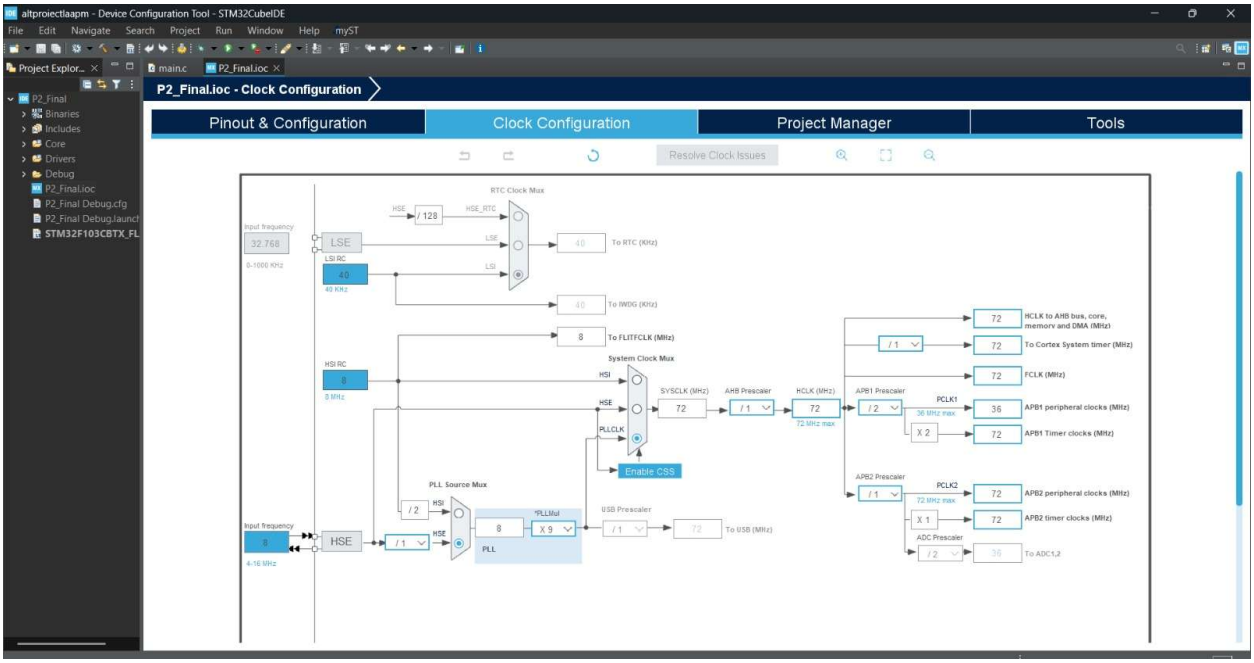
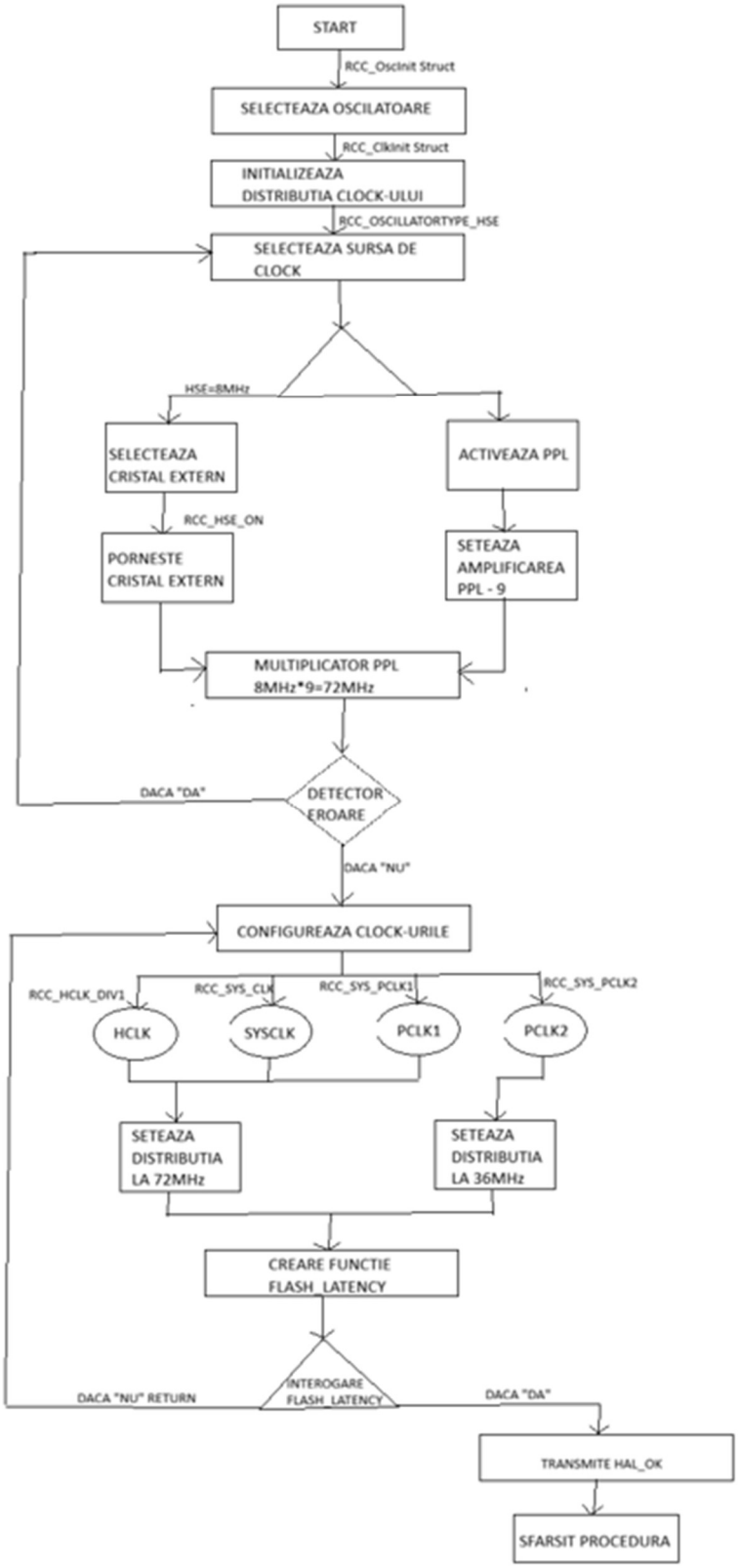


figura 12 – configurarea arborelui de ceas (clock configuration)

figură – configurarea frecvenței procesorului



Pin ...	Signal o...	GPIO o...	GPIO m...	GPIO P...	Maximu...	User La...	Modified
PA1	n/a	n/a	Input mo...	No pull-...	n/a	B0	☑
PA2	n/a	n/a	Input mo...	No pull-...	n/a	B1	☑
PA3	n/a	n/a	Input mo...	No pull-...	n/a	B2	☑
PA4	n/a	n/a	Input mo...	No pull-...	n/a	B3	☑
PA5	n/a	n/a	Input mo...	No pull-...	n/a	B4	☑
PA6	n/a	n/a	Input mo...	No pull-...	n/a	B5	☑
PA7	n/a	n/a	Input mo...	No pull-...	n/a	B6	☑
PA8	n/a	n/a	Input mo...	No pull-...	n/a	B7	☑
PA9	n/a	n/a	Input mo...	No pull-...	n/a	PRG	☑

figura 13 – pinout PA1-PA8 (B0-B7) și PA9 (PRG)

PB0	n/a	Low	Output ...	No pull-...	Low	LED0	☑
PB1	n/a	Low	Output ...	No pull-...	Low	LED1	☑
PB2	n/a	Low	Output ...	No pull-...	Low	LED2	☑
PB3	n/a	Low	Output ...	No pull-...	Low	LED3	☑
PB4	n/a	Low	Output ...	No pull-...	Low	LED4	☑
PB5	n/a	Low	Output ...	No pull-...	Low	LED5	☑
PB6	n/a	Low	Output ...	No pull-...	Low	LED6	☑
PB7	n/a	Low	Output ...	No pull-...	Low	LED7	☑
PB8	n/a	Low	Output ...	No pull-...	Low	PCDA0	☑

figura 14 – pinout PB0-PB7 (LED0-7) și PB8 (PCDA0)

PB9	n/a	Low	Output ...	No pull-...	Low	PCDA1	☑
PB10	n/a	Low	Output ...	No pull-...	Low	PCDA2	☑
PB11	n/a	Low	Output ...	No pull-...	Low	PCDA3	☑
PB12	n/a	Low	Output ...	No pull-...	Low	PCDA4	☑
PB13	n/a	Low	Output ...	No pull-...	Low	PCDA5	☑
PB14	n/a	Low	Output ...	No pull-...	Low	PCDA6	☑
PB15	n/a	Low	Output ...	No pull-...	Low	PCDA7	☑

figura 15 – pinout PB9-PB15 (PCDA1-7)

8. Subsistemul DSP – descriere conceptuală/funcțională

8.1. Structura programului:

Programul este scris în limbaj de asamblare ADSP-21xx și este organizat în secțiuni de memorie conform arhitecturii Harvard:

- **buf_var1 (DM):** buffer de recepție rx_buf[3] — stare + canal stâng + canal drept
- **buf_var2 (DM):** buffer de transmisie tx_buf[3] — comandă + canal stâng + canal drept
- **buf_var3 (DM):** cele 13 cuvinte de inițializare a codec-ului AD1847
- **data1 (DM):** variabilele programului (Flag_CDA, cmd_word, cmd_valida, id_functie, param_id, canal, x_in, y_out, PF_input, PF_output, stat_flag)
- **interrupts (PM):** tabela vectorilor de întrerupere
- **seg_code (PM):** codul principal

8.2. Inițializarea sistemului:

SPORT0: Se configurează autobuffering-ul, modul multicanal pe 16 biți, cu codec-ul furnizând ceasul. Bufferele circulare sunt configurate: i5/l5 pentru rx_buf, i6/l6 pentru tx_buf, i3/l3 pentru init_cmds.

Codec AD1847: Se trimit secvențial 13 cuvinte de comandă prin SPORT0 TX. Se așteaptă finalizarea auto-calibrării (bitul ACI trece 0→1→0 în registrul de stare). Se dezactivează mute-ul pe canalele de ieșire.

Porturi PF: PF0–PF3 configurate ca intrări, PF4–PF7 ca ieșiri (Prog_Flag_Comp_Sel_Ctrl = 0x00F0).

Întreruperi: imask = b#0010100001 activează Timer (bitul 9), SPORT0 RX (bitul 5) și IRQL0 (bitul 0).

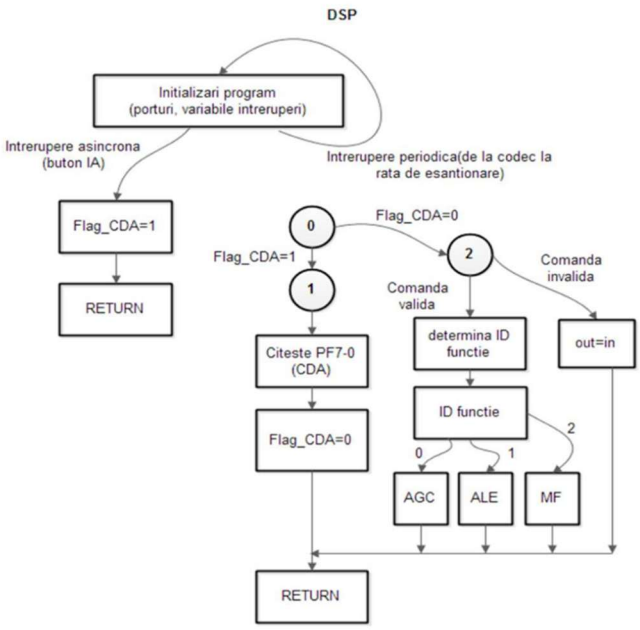
8.3. Întreruperile active:

IRQL0 (isr_ia): Întreruperea asincronă de la butonul IA. Setează Flag_CDA = 1 pentru a semnala mașinii de stări că o comandă nouă este disponibilă.

Timer (isr_periodic): Rulează mașina de stări DSP la fiecare declanșare.

SPORT0 RX (input_samples): La fiecare eșantion audio recepționat de la codec: citește rx_buf+2 în x_in, scrie y_out în tx_buf+1 și tx_buf+2 (ambele canale), realizează testul I/O și actualizează porturile PF.

8.4. Mașina de stări DSP:



STARE 0 – RECEPȚIONARE/DECIZIE: Verifică dacă Flag_CDA != 0 (primește comandă nouă), sare la starea 1. Dacă Flag_CDA == 0, sare la starea 2.

STARE 1 – CITIREA COMENZII: Citește Prog_Flag_Data, folosind masca 0x00FF și salvează rezultatul în cmd_word. Resetează Flag_CDA = 0. Execută returnare (rti).

STARE 2 – VALIDARE ȘI EXTRAGERE A PARAMETRILOR + EXECUȚIE:

- validare: validează mesajul oferit (cmd_word & 0x00C0) == 0x00C0?, dacă nu, atunci efectuează returnare (rti).
- extragere CDA: primește id_functie (0 = AGC, 1 = ALE, 2 = MF)
- extragere parametru: param_id (0-7) din cmd_word & 0x0007
- extragere canal: canal (0 sau 1) din cmd_word & 0x0008
- rutare(id_functie): 0 - do_agc, 1 - do_ale, 2 - do_mf

CODUL ÎN LIMBAJ DE ASAMBLARE A DSP:

```
#include "def2181.h"

#define PORT_OUT 0xFF
#define PORT_IN  0x1FF

.SECTION/DM buf_var1;
.var/circ rx_buf[3];
.SECTION/DM buf_var2;
.var/circ tx_buf[3] = 0xc000, 0x0000, 0x0000;
.SECTION/DM buf_var3;
.var init_cmds[13] = 0xc002,
            0xc102,
            0xc288,
            0xc388,
            0xc488,
            0xc588,
            0xc680,
            0xc780,
            0xc85c,
            0xc909,
            0xca00,
            0xcc40,
            0xcd00;

.SECTION/DM data1;
.var stat_flag;

.var PF_input;
.var PF_output;

.var Flag_CDA;
.var cmd_word;
.var cmd_valida;
```

```
.var id_functie;
.var x_in;
.var y_out;
.var param_id;
.var canal;

.SECTION/PM pm_da;

.SECTION/PM interrupts;
    jump start;       rti; rti; rti;    /*00 reset */
    rti;              rti; rti; rti;    /*04 IRQ2 */
    rti;              rti; rti; rti;    /*08 IRQL1 */
    jump isr_ia;      rti; rti; rti;    /*0C IRQL0 */

    ar = dm(stat_flag);                 /*10 SPORT0 tx */
    ar = pass ar;
    if eq rti;
    jump next_cmd;

    jump input_samples;                 /*14 SPORT0 rx*/
    rti;              rti; rti;

    rti;              rti; rti; rti;    /*18 IRQE */
    rti;              rti; rti; rti;    /*1C BDMA */
    rti;              rti; rti; rti;    /*20 SPORT1 tx / IRQ1 */
    rti;              rti; rti; rti;    /*24 SPORT1 rx / IRQ0 */
    rti;              rti; rti; rti;
    rti;              rti; rti; rti;

.SECTION/PM seg_code;

start:

    ax0 = b#0000100000000000;
    dm(Sys_Ctrl_Reg) = ax0;
```

```
i5 = rx_buf;
l5 = LENGTH(rx_buf);
i6 = tx_buf;
l6 = LENGTH(tx_buf);
i3 = init_cmds;
l3 = LENGTH(init_cmds);

m1 = 1;
m5 = 1;

ax0 = b#0000110011010111;
dm(Sport0_Autobuf_Ctrl) = ax0;

ax0 = 0;
dm(Sport0_Rfsdiv) = ax0;

ax0 = 0;
dm(Sport0_Sclkdiv) = ax0;

ax0 = b#1000011000001111;
dm(Sport0_Ctrl_Reg) = ax0;

ax0 = b#0000000000000111;
dm(Sport0_Tx_Words0) = ax0;

ax0 = b#0000000000000111;
dm(Sport0_Tx_Words1) = ax0;

ax0 = b#0000000000000111;
dm(Sport0_Rx_Words0) = ax0;

ax0 = b#0000000000000111;
dm(Sport0_Rx_Words1) = ax0;
ax0 = b#0001100000000000;
```

```
    dm(Sys_Ctrl_Reg) = ax0;

    ifc = b#00000011111110;
    nop;

    icntl = b#00010;
    mstat = b#1100000;

    ax0 = 1;
    dm(stat_flag) = ax0;

    ena ints;
    imask = b#0001000001;

    ax0 = dm(i6, m5);
    tx0 = ax0;

check_init:
    ax0 = dm(stat_flag);
    af = pass ax0;
    if ne jump check_init;
    ay0 = 2;

check_aci1:
    ax0 = dm(rx_buf);
    ar = ax0 and ay0;
    if eq jump check_aci1;

check_aci2:
    ax0 = dm(rx_buf);
    ar = ax0 and ay0;
    if ne jump check_aci2;
    idle;
```

```
ay0 = 0xbf3f;
ax0 = dm(init_cmds + 6);
ar = ax0 and ay0;
dm(tx_buf) = ar;
idle;

ax0 = dm(init_cmds + 7);
ar = ax0 and ay0;
dm(tx_buf) = ar;
idle;

ifc = b#00000011111110;
nop;

imask = b#0000100000;

si = 0xFFFF;
dm(Dm_Wait_Reg) = si;

si = 0x00f0;
dm(Prog_Flag_Comp_Sel_Ctrl) = si;

ax0 = 0;
dm(PF_input)    = ax0;
dm(PF_output)   = ax0;
dm(Flag_CDA)    = ax0;
dm(cmd_word)    = ax0;
dm(cmd_valida)  = ax0;
dm(id_functie)  = ax0;
dm(x_in)        = ax0;
dm(y_out)       = ax0;
dm(param_id)    = ax0;
dm(canal)       = ax0;

ena ints;
```

```
    imask = b#0000100001;

    imask = b#0010100000;

main_loop:
    idle;
    jump main_loop;

isr_ia:
    ax0 = 1;
    dm(Flag_CDA) = ax0;
    rti;

input_samples:
    ena sec_reg;

    sr1 = dm(rx_buf + 2);
    dm(x_in) = sr1;

    si = IO(PORT_IN);
    IO(PORT_OUT) = si;

    ax0 = dm(Prog_Flag_Data);
    ay0 = 0x000F;
    ar = ax0 and ay0;
    dm(PF_input) = ar;



starea_0:
    ax0 = dm(Flag_CDA);
    af = pass ax0;
    if ne jump starea_1;
    jump starea_2;
```

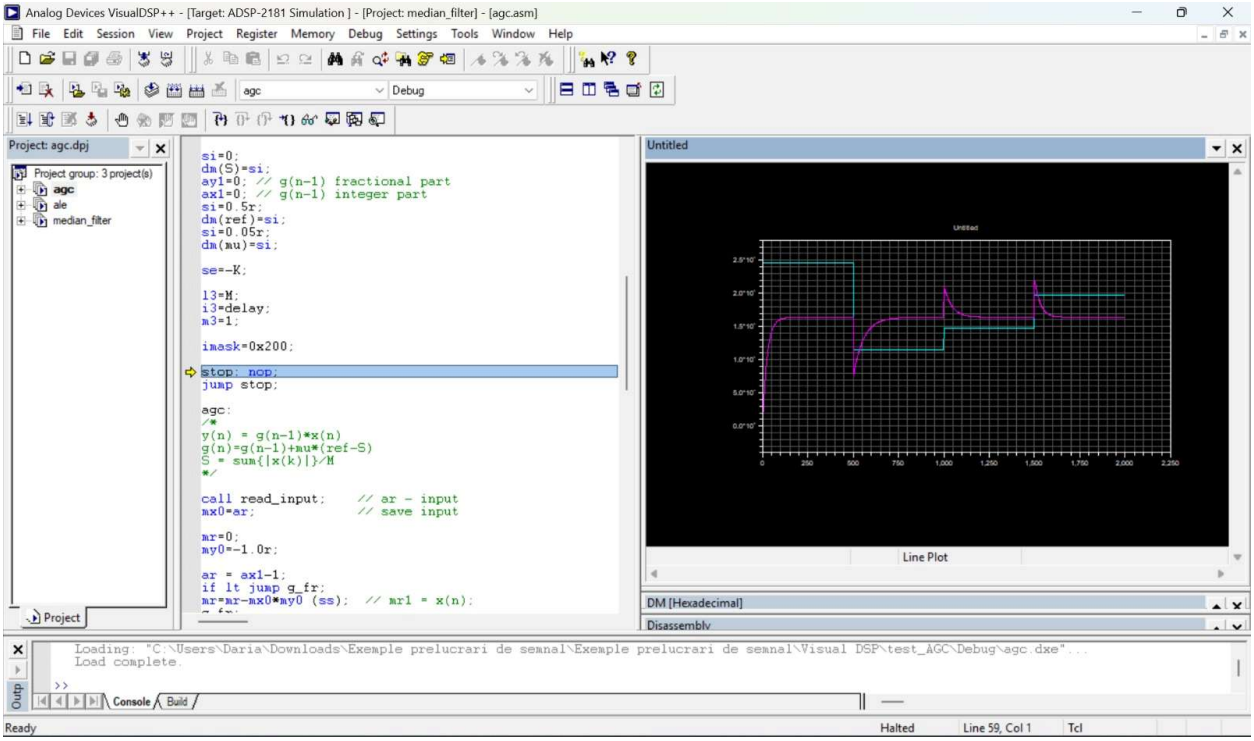
```
starea_1:
    ax0 = dm(Prog_Flag_Data);
    ay0 = 0x00FF;
    ar = ax0 and ay0;
    dm(cmd_word) = ar;

    ax0 = 0;
    dm(Flag_CDA) = ax0;

    jump output_and_return;

starea_2:
    ax0 = dm(cmd_word);
    ay0 = 0x00C0;
    ar = ax0 and ay0;
    af = ar - ay0;
    if ne jump cmd_invalida;

    ax0 = 1;
    dm(cmd_valida) = ax0;

    ax0 = dm(cmd_word);
    ay0 = 0x0030;
    ar = ax0 and ay0;
    sr = lshift ar by -4 (lo);
    dm(id_functie) = sr0;

    ax0 = dm(cmd_word);
    ay0 = 0x0008;
    ar = ax0 and ay0;
    sr = lshift ar by -3 (lo);
    dm(canal) = sr0;

    ax0 = dm(cmd_word);
    ay0 = 0x0007;
```

```
    ar = ax0 and ay0;
    dm(param_id) = ar;

    ax0 = dm(id_functie);
    none = pass ax0;
    if eq jump do_agc;

    ay0 = 1;
    ar = ax0 - ay0;
    if eq jump do_ale;
    ay0 = 2;
    ar = ax0 - ay0;
    if eq jump do_mf;

    jump do_bypass;

cmd_invalida:
    ax0 = 0;
    dm(cmd_valida) = ax0;
    jump do_bypass;


do_agc:
    /* TODO*/
    jump do_bypass;

do_ale:
    /* TODO*/
    jump do_bypass;

do_mf:
    /* TODO*/
    jump do_bypass;
```

```
do_bypass:
    ax0 = dm(x_in);
    dm(y_out) = ax0;
    jump output_and_return;


output_and_return:
    ax0 = dm(y_out);
    dm(tx_buf + 1) = ax0;
    dm(tx_buf + 2) = ax0;

    ar = dm(PF_output);
    sr = lshift ar by 4 (hi);
    ax0 = sr1;
    dm(Prog_Flag_Data) = ax0;

    rti;

next_cmd:
    ena sec_reg;
    ax0 = dm(i3, m1);
    dm(tx_buf) = ax0;
    ax0 = i3;
    ay0 = init_cmds;
    ar = ax0 - ay0;
    if gt rti;
    ax0 = 0xaf00;
    dm(tx_buf) = ax0;
    ax0 = 0;
    dm(stat_flag) = ax0;
    rti;
```

8.5. Simulări DSP (VisualDSP++):

Simulare AGC:



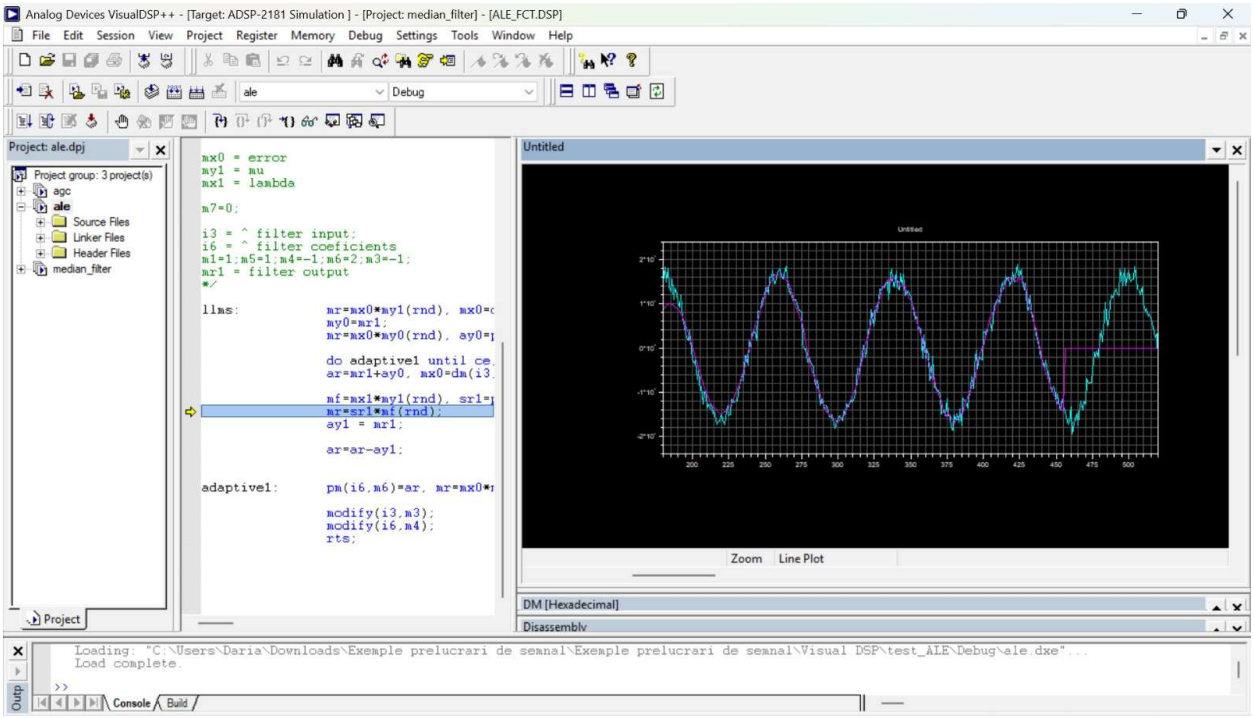

```
#include   "def2181.h"

#define M 4
#define K 2

.SECTION/DM                                data1;

.var/circ rx_buf[3];
.var/circ tx_buf[3];

.var/circ delay[M];
.var S;
.var ref;
.var mu;



/*** Interrupt Vector Table ***/
.SECTION/PM    interrupts;
```

```
                    jump start;  rti; rti; rti;    /*00: reset */
jump agc;       rti; rti; rti;    /*04: IRQ2 - placa EZKIT*/
rti;       rti; rti; rti;    /*08: IRQL1 */
rti;       rti; rti; rti;    /*0c: IRQL0 */
  rti;       rti; rti; rti;    /*10: SPORT0 tx */
  rti;            rti; rti; rti;    /*14: SPORT1 rx */
rti;       rti; rti; rti;    /*18: IRQE */
rti;       rti; rti; rti;    /*1c: BDMA */
rti;       rti; rti; rti;    /*20: SPORT1 tx or IRQ1 */
rti;       rti; rti; rti;    /*24: SPORT1 rx or IRQ0 */
rti;       rti; rti; rti;    /*28: timer */
rti;       rti; rti; rti;    /*2c: power down */


.SECTION/PM                    seg_code;
/*******************************************************************************
 *
 *  ADSP 2181 intialization
 *
 *******************************************************************************/
start:
si=0;
dm(S)=si;
ay1=0; // g(n-1) fractional part
ax1=0; // g(n-1) integer part
si=0.5r;
dm(ref)=si;
si=0.05r;
dm(mu)=si;

se=-K;

l3=M;
i3=delay;
m3=1;

imask=0x200;

stop: nop;
jump stop;

agc:
/*
y(n) = g(n-1)*x(n)
```

```
g(n)=g(n-1)+mu*(ref-S)
S = sum{|x(k)|}/M
*/
call read_input;                    // ar - input
mx0=ar;                                     // save input

mr=0;
my0=-1.0r;

ar = ax1-1;
if lt jump g_fr;
mr=mr-mx0*my0 (ss);  // mr1 = x(n);
g_fr:
my1=ay1;
mr=mr+mx0*my1 (rnd); // mr1= y(n) = interger_part_g(n-1)*x(n-1)+fractional_part_g(n-1)*x(n) <1

dm(tx_buf+2)=mr1;                   // write output

//
ar = abs mr1;
sr=ashift ar (hi); // sr1 - scaled input
dm(i3,m3)=sr1;                      // update delay line

// compute average
cntr=M-1;
mr=0, mx0=dm(i3,m3);
do sum until ce;
sum: mr=mr-mx0*my0(ss), mx0=dm(i3,m3);
mr=mr-mx0*my0(rnd);
if mv sat mr;

dm(S)=mr1;

ax0=dm(ref);
ay0=dm(S);
ar=ax0-ay0; // ar=ref-S

// ay1= fractional part of g(n-1);
// ax1 = integer part of g(n-1)

my1=ar;
mx1=dm(mu);

mr=mx1*my1 (ss);                    // mr1 = mu*(ref-S)
```

```
ar=mr1+ay1;                                    // ar = g(n-1)+ mu*(ref-S) = g(n)
af = pass ar;                                  // ar < 0 -> overflow g(n)>1, update integer and fractional
parts
if ge jump cont;
// gain correction: ax1=1 -> ax1=1, ay1=0; ax1=1 -> ax1=0; ay1=0x7FFF
af = ax1-1;
if lt jump cor1;
// ax1=1 -> ay1=1.0, ax1=0;
ax1=0;
ay1=0x7fff;
rti;
// ax1=0 -> ay1=0.0, ax1=1
cor1:
ax1=1;
ay1=0;
rti;

cont:
ay1=ar;
rti;
//////////////////////////////////////////////////
// Input, Output functions
//////////////////////////////////////////////////

read_input:
                                    ar=dm(rx_buf+2);
                                    rts;
write_output:
                                    dm(tx_buf+2)=mr1;
                                    rts;
```

Simulare ALE:




```
/* Acest program implementeaza rutinele de filtrare FIR si de adaptare
a coeficientilor conform algoritmului LLMS (Leaky LMS).
*/


.section/pm program;

.global fir;
.global llms;


// ----------------FIR Subroutine--------------------------
/*
i3 = ^ filter input;
i6 = ^ filter coeficients
m1=1;m5=1;
mr1 = filter output
*/

fir:        mr=0, mx0=dm(i3,m1), my0=pm(i6,m5);
            do sop until ce;
sop:            mr=mr+mx0*my0(ss), mx0=dm(i3,m1), my0=pm(i6,m5);
            mr=mr+mx0*my0(rnd);
```

```
        if mv sat mr;
        rts;

//----------------LLMS Modify Subroutine-----------------------

// h(k) = h(k)*(1-mu*lambda) + mu*x(n-k)*e(n)
/*

mx0 = error
my1 = mu
mx1 = lambda

m7=0;

i3 = ^ filter input;
i6 = ^ filter coeficients
m1=1;m5=1;m4=-1;m6=2;m3=-1;
mr1 = filter output
*/

llms:
mr=mx0*my1(rnd), mx0=dm(i3,m1);   // mr1 = error * mu , mx0= x(n-k)
        my0=mr1;                                                    // my0 = error*mu
mr=mx0*my0(rnd), ay0=pm(i6,m5);      // mr1 = x(n-k)*mu*error, ay0= h(k)

    do adaptive1 until ce;
   ar=mr1+ay0, mx0=dm(i3,m1), ay0=pm(i6,m4); //ar = h(k) +x(n-k)*mu*error

    mf=mx1*my1(rnd), sr1=pm(i6,m7);            // mf= mu*lambda, sr1=h(k)
                        mr=sr1*mf(rnd);                  // mr1=h(k)*mu*lambda
                        ay1 = mr1;                              // ay1=h(k)*mu*lambda

        ar=ar-ay1;                                              //ar = h(k)-h(k)*mu*lambda+x(n-
k)*mu*error


adaptive1:    pm(i6,m6)=ar, mr=mx0*my0(rnd);

                                modify(i3,m3);
        modify(i6,m4);
        rts;
```

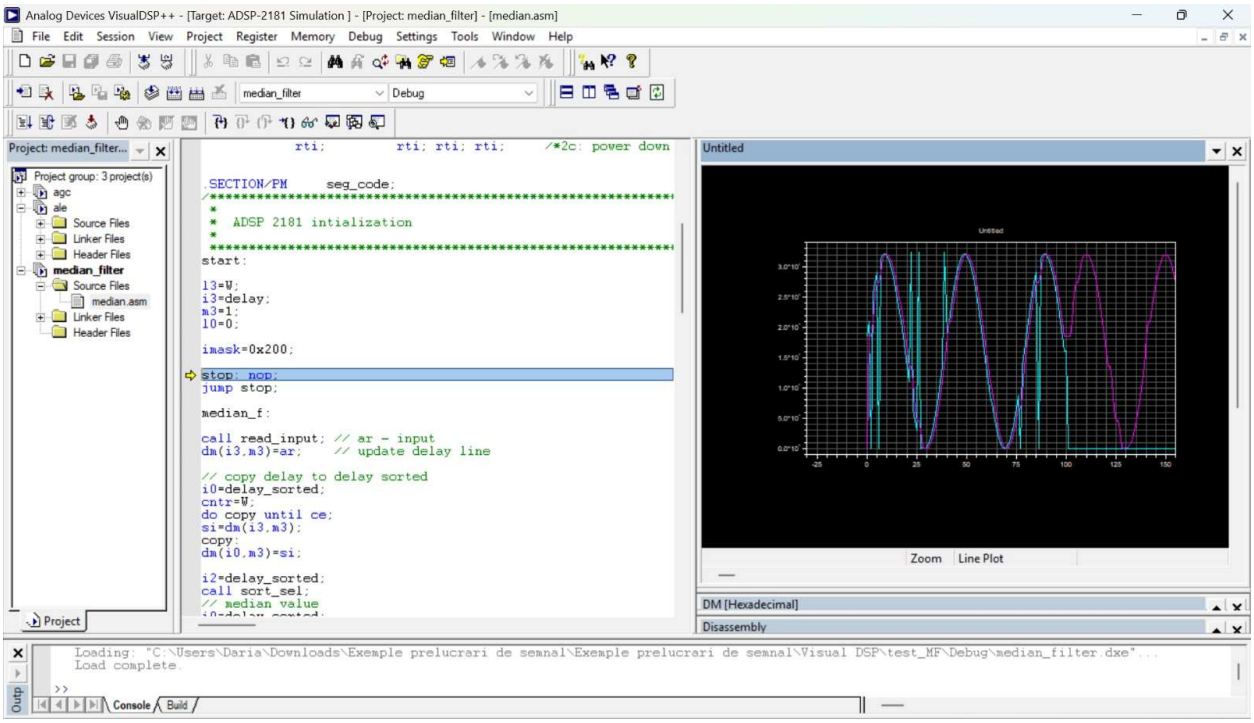
```
-----------------------------------------------------------
/*****************************************************************
 *
 *  Acest program testeaza filtrul ALE :
 *
 *  - tabelul de intreruperi specific ADSP2181
 *  - rutinele de servire a intreruperilor
 *****************************************************************/


.extern fir;
.extern lms;
.extern llms;
.extern read_input;
.extern write_output;
#include "ale.h"                    //definirea constantelor

.section/dm data1;

.var/circ       fir_d[n];       //linia de intirziere a FIR
.var/circ       input[n];       //linia de intirziere pentru intrare

.section/pm data2;

.var/circ       hh[n];          //coeficientii FIR

.section/dm data1;

.var/circ rx_buf[3];
.var/circ tx_buf[3];
.global rx_buf;
.global tx_buf;
.var save;

//Tabela vectorilor de intrerupere

.section/pm IVreset;

    jump start;  rti; rti; rti;    // 00: reset
        jump input_samples;
                        rti; rti; rti;    // 04: IRQ2
    rti;      rti; rti; rti;    // 08: IRQL1
    rti;      rti; rti; rti;    // 0c: IRQL0
    rti;            rti;rti;rti;    // 10: SPORT) tx
```

```
      rti;      rti; rti; rti;    // 14: SPORT1 rx
      rti;       rti; rti; rti;    // 18: IRQE
      rti;       rti; rti; rti;    // 1c: BDMA
      rti;       rti; rti; rti;    // 20: SPORT1 tx or IRQ1
      rti;       rti; rti; rti;    // 24: SPORT1 rx or IRQ0
      rti;       rti; rti; rti;    // 28: timer
      rti;       rti; rti; rti;    // 2c: power down

.section/pm program;
start:
            //initializari registre

            m1=1;
            m5=1;
            si=0;
            i2=input;
            l2=n;
            i4=input+delay;
            l4=n;
            i3=fir_d;
            l3=n;
            i6=hh;
            l6=n;

            //initializare linii intirziere si coeficienti

            cntr=n;
            do init_buf until ce;
            dm(i2,m1)=si;
            dm(i3,m1)=si;
init_buf:
            pm(i6,m5)=si;

  imask =0x200;;

      //Asteptare intreruperi

wait:   jump wait;

//Rutina de servire a intreruperii de esantionare IRQ2}

input_samples:
   ena sec_reg;          // se utilizeaza setul secundar de registre
            ena ar_sat;
```

```
        call read_input;        // in ar - esantionul de intrare
        dm(save)=ar;

        dm(i2,m1)=ar;           // actualizeaza linia de intirziere de intrare
        ar=dm(i4,m5);
        dm(i3,m1)=ar;           // actualizeaza linia de intirziere a FIR

        //calculeaza iesirea FIR

        cntr=n-1;
        call fir;           // mr1 - iesirea filtrului

        //scrie iesirea - semnalul "curatat"

        call write_output;      //in mr1 - valoarea dorita pentru scriere

        //calcul eroare

        ar=dm(save);    // in ar - esantionul de intrare
        ay0=ar;
        ar=ay0-mr1;             // in ar - eroarea
        mx0=ar;
        cntr=n;
        my1=mu;
        m4=-1;
        m6=2;
        m3=-1;
        //call lms;             // adapteaza coeficientii FIR
        m7=0;
        mx1=lambda;
        call llms;
    dis sec_reg;        // revine la setul primar de registre
    rti;
-----------------------------------------------------
.section/pm program;

.global read_input;
.global write_output;

.extern rx_buf;
.extern tx_buf;

read_input:
```

```
        ar=dm(rx_buf+2);
        rts;
write_output:
        dm(tx_buf+2)=mr1;
        rts;
```

Simulare MF:




```
#include   "def2181.h"

#define W 3
#define K 1

.SECTION/DM             data1;

.var/circ rx_buf[3];
.var/circ tx_buf[3];

.var/circ delay[W];
.var delay_sorted[W];
```

```
/*** Interrupt Vector Table ***/
.SECTION/PM    interrupts;
                                jump start;  rti; rti; rti;    /*00: reset */
          jump median_f;       rti; rti; rti;    /*04: IRQ2 - placa EZKIT*/
          rti;       rti; rti; rti;    /*08: IRQL1 */
          rti;       rti; rti; rti;    /*0c: IRQL0 */
                     rti;       rti; rti; rti;    /*10: SPORT0 tx */
                     rti;            rti; rti; rti;    /*14: SPORT1 rx */
          rti;       rti; rti; rti;    /*18: IRQE */
          rti;       rti; rti; rti;    /*1c: BDMA */
          rti;       rti; rti; rti;    /*20: SPORT1 tx or IRQ1 */
          rti;       rti; rti; rti;    /*24: SPORT1 rx or IRQ0 */
          rti;       rti; rti; rti;    /*28: timer */
          rti;       rti; rti; rti;    /*2c: power down */

.SECTION/PM              seg_code;
/*****************************************************************************
 *
 *  ADSP 2181 intialization
 *
 *****************************************************************************/
start:

l3=W;
i3=delay;
m3=1;
l0=0;

imask=0x200;

stop: nop;
jump stop;

median_f:

call read_input; // ar - input
dm(i3,m3)=ar;   // update delay line

// copy delay to delay sorted
i0=delay_sorted;
cntr=W;
do copy until ce;
si=dm(i3,m3);
copy:
```

```
dm(i0,m3)=si;

i2=delay_sorted;
call sort_sel;
// median value
i0=delay_sorted;
m0=K;
modify(i0,m0);
ar=dm(i0,m3);
dm(tx_buf+2)=ar;

rti;

////////////////////////////////////////////////////////////////////
// sortare prin selectie
sort_sel:
/*
	for (i=0; i<W; i++)
		{
			min=i;
			for (j=i+1; j<W; j++)
				{
				if A[j]<A[min] min=j;
				}
			tmp=A[i];
			A[i]=A[min];
			A[min]=tmp;
		}
*/
// sorting

// m0 - min
// m1 - (i)
// m2 - (i+1)

// i2 pointer to A

m3=1;

l0=0;
l1=0;

cntr=W;
//	for (i=0; i<W-1; i++)
```

```
do loop_i until ce;
ax0=W;
ay0=cntr;
ar=ax0-ay0; //i
m0=ar;                          // m0 = min
m1=ar;              // m1 = i
ar=ar+1;
m2=ar;              // m2 = i+1
ay0=m1;
ar=ax0-ay0;
ar=ar-1;            // # of iteration in loop_j

if eq jump loop_i;

cntr=ar;

i0=i2;
modify(i0,m2); // i0=A[i+1]

// for (j=i+1; j<W; j++)
            do loop_j until ce;
            //if A[j]<A[min] min=j;
            ax0=dm(i0,m3); // ax0= A[j]
            i1=i2;
            modify(i1,m0);
            ay0=dm(i1,m3); // ay0=A[min]
            ar=ax0-ay0;
            if gt jump loop_j;
            ax0=W;
            ay0=cntr;
            ar=ax0-ay0; //j
            m0=ar;                          // min = j
            loop_j: nop;
/*
tmp=A[i];
A[i]=A[min];
A[min]=tmp;
*/
i1=i2;
modify(i1,m1);
ax1=dm(i1,m3);            // ax1 = tmp = A[i];
i1=i2;
modify(i1,m0);
ay1=dm(i1,m3);            // ay1=A[min]
```

```
i1=i2;
modify(i1,m1);
dm(i1,m3)=ay1;          // A[i]=A[min]
i1=i2;
modify(i1,m0);
dm(i1,m3)=ax1;          // A[min]=tmp
loop_i: nop;

rts;
//////////////////////////////////////////////////
// Input, Output functions
//////////////////////////////////////////////////

read_input:
        ar=dm(rx_buf+2);
        rts;
write_output:
        dm(tx_buf+2)=mr1;
        rts;
```

9. Simulări MATLAB:

Prin simulările MATLAB putem observa comportamentul așteptat al funcțiilor de procesare care vor fi implementate. Acestea servesc pentru validarea implementărilor viitoare pe DSP.

figura – simulare AGC

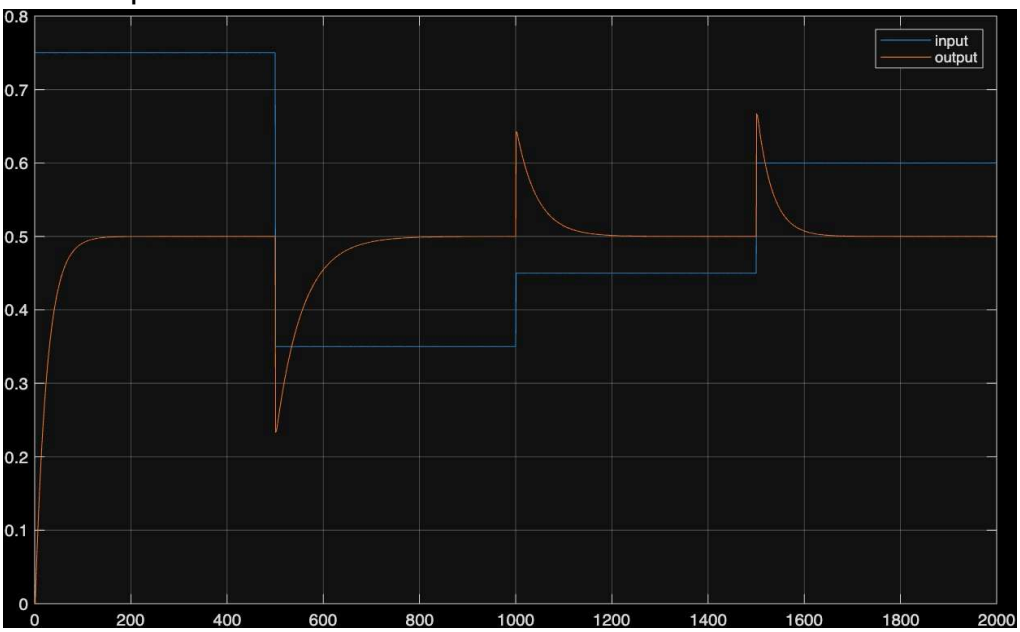


figura – simulare ALE

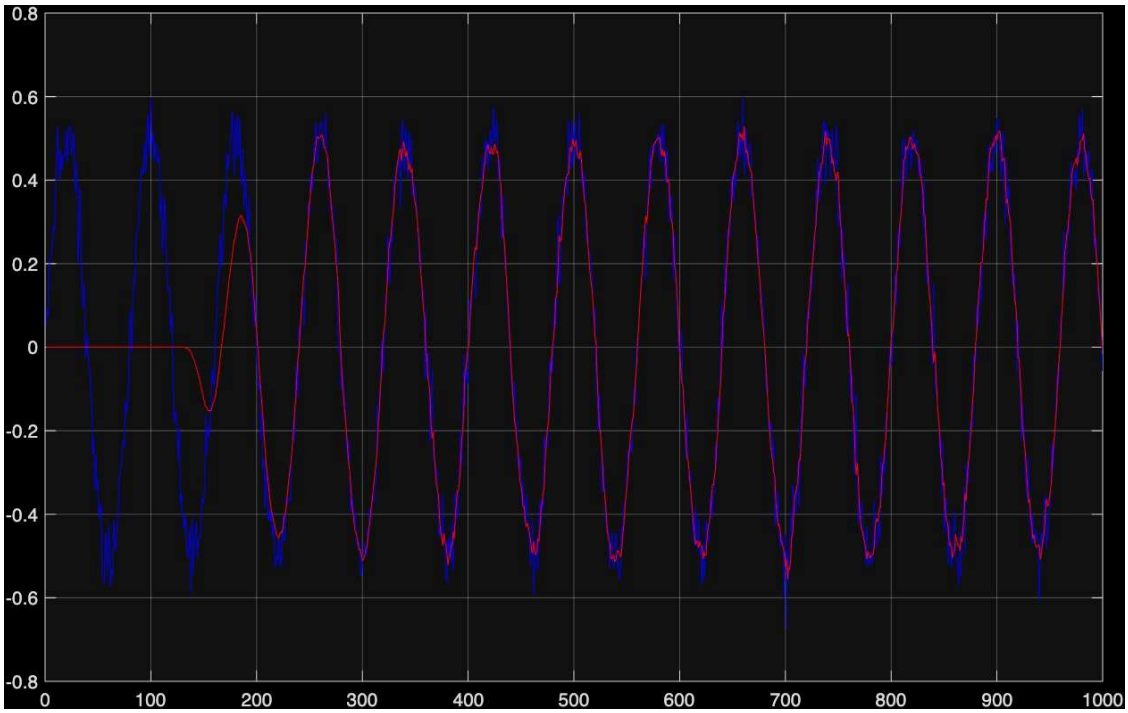


figura – simulare MF (cu semnale separate)

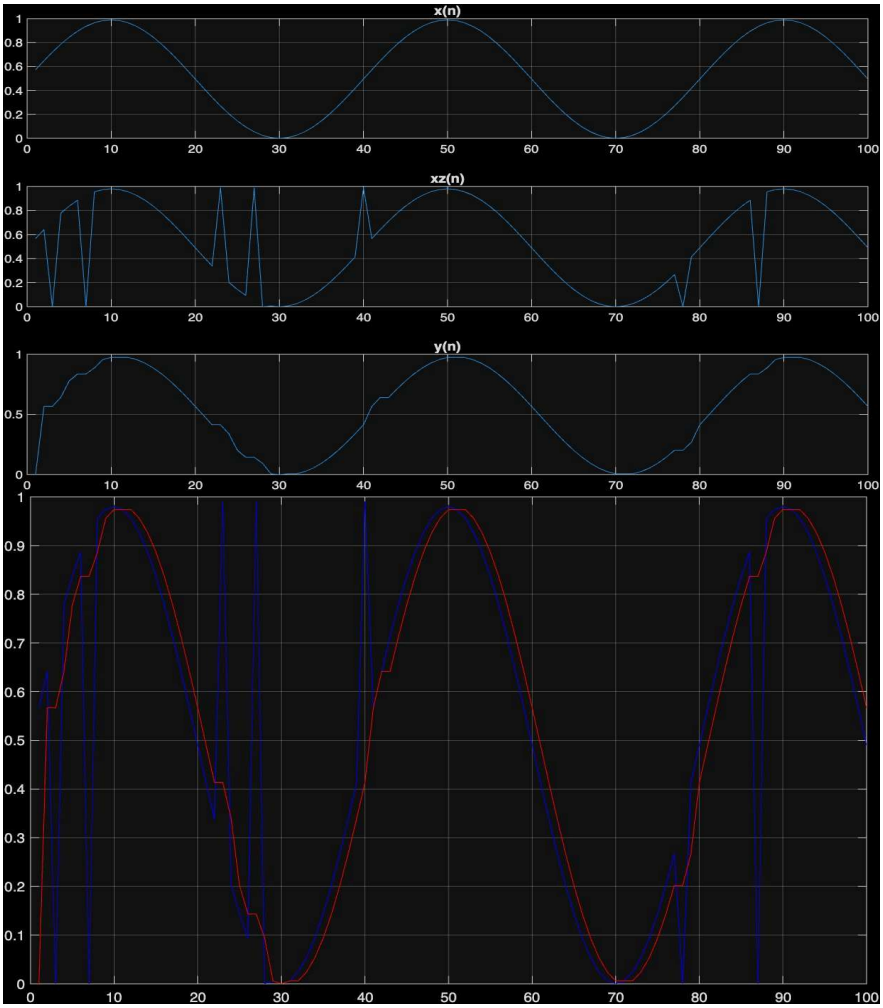


figura – simulare MF (cu semnalele de intrare și ieșire suprapuse)